

Image

# 大语言模型工程实践

从模型原理到企业知识助手

作者：夏同

时间：May 14, 2026

---

# 目录

|                                               |    |
|-----------------------------------------------|----|
| 习题索引                                          | ii |
| <br>                                          |    |
| 第一部分 理解与使用大语言模型                               | 1  |
| 第 1 章 走近大语言模型：能力边界与主案例                        | 2  |
| 第 2 章 Transformer 核心机制：Attention、FFN、Norm 与损失 | 7  |
| 第 3 章 从库到模型：Transformers 使用、推理与调试基础           | 13 |
| 第 4 章 提示工程与上下文管理：Prompting、采样与长期记忆            | 17 |
| <br>                                          |    |
| 第二部分 训练与改造模型                                  | 21 |
| 第 5 章 微调路线图：SFT、PEFT、LoRA、QLoRA 与 PEFT 实战     | 22 |
| 第 6 章 数据工程与继续预训练：数据构建、样本拼接、增量预训练              | 28 |
| <br>                                          |    |
| 第三部分 构建知识系统                                   | 35 |
| 第 7 章 RAG 基础：检索增强问答系统的最小闭环                    | 36 |
| 第 8 章 文档理解与检索优化：PDF、表格、召回增强与 GraphRAG         | 41 |
| 第 9 章 评测体系：模型、系统与应用层评测                        | 46 |
| <br>                                          |    |
| 第四部分 对齐、优化与规模化工程                              | 52 |
| 第 10 章 对齐与强化学习：RLHF、PPO 与 NLP 中的 RL           | 53 |
| 第 11 章 推理与性能优化：显存、量化、吞吐与服务稳定性                 | 58 |
| 第 12 章 分布式训练与大规模工程：并行策略、训练框架、成本与排障            | 63 |

---

## 习题索引

|      |               |    |
|------|---------------|----|
| 1.1  | 能力分层判断        | 6  |
| 1.2  | 先判层再判手段       | 6  |
| 2.1  | 长上下文答偏        | 11 |
| 2.2  | 损失与业务目标       | 11 |
| 3.1  | 输出失控先看哪一层     | 15 |
| 3.2  | 复盘最小日志        | 16 |
| 4.1  | 提示修复          | 19 |
| 4.2  | 记忆设计          | 19 |
| 5.1  | 先改提示还是上 LoRA  | 26 |
| 5.2  | 什么时候上 QLoRA   | 26 |
| 6.1  | 样本去向判断        | 33 |
| 6.2  | 继续预训练触发条件     | 33 |
| 7.1  | 检索失败还是生成失败    | 40 |
| 7.2  | 为什么先做最小闭环     | 40 |
| 8.1  | 文档解析失败        | 44 |
| 8.2  | 何时考虑 GraphRAG | 44 |
| 9.1  | 指标设计          | 50 |
| 9.2  | 测试集污染         | 50 |
| 10.1 | KL 的作用        | 57 |
| 10.2 | 奖励黑客风险        | 57 |
| 11.1 | 显存瓶颈定位        | 62 |
| 11.2 | 量化还是批处理       | 62 |
| 12.1 | 并行策略选择        | 67 |
| 12.2 | 训练故障归因        | 67 |

---

## 第一部分

# 理解与使用大语言模型

# 第 1 章 走近大语言模型：能力边界与主案例

## 内容提要

❑ **本章核心问题：**在企业知识助手这个主案例里，大语言模型究竟是什么，它能做什么、不能做什么，又为什么不能被直接当成一个完整业务系统？

❑ **主案例推进：**定义企业知识助手的第一版

目标，明确模型层、任务层、系统层三类能力边界。

❑ **读完后能做什么：**遇到一个 LLM 问题时，先判断它属于模型能力、任务能力还是系统能力，再决定优先修哪里。

## 1.1 学习目标

- 理解大语言模型在工程系统中的角色，而不是把它当成抽象名词。
- 区分模型能力、任务能力与系统能力的边界。
- 建立全书贯穿案例“企业知识助手”的目标、约束与演进路线。

## 1.2 问题背景

员工在企业内部助手里提问“出差报销上限是多少”，系统给出了一个流畅答案，但没有引用依据。制度更新一周后，它仍沿用旧说法；换个部门再问，回答又变成另一个版本。这个例子说明，业务系统真正关心的从来不只是“模型会不会说话”，而是“答案从哪里来、为什么可信、出了错该怎么定位”。

过去几年里，大语言模型从“会聊天的模型”逐步演化为“可编排的工程组件”。对工程读者来说，真正重要的问题不再是“模型会不会生成一段漂亮文字”，而是“模型怎样被稳定接入系统”，以及“系统失效时应该先排查模型、数据、提示词，还是检索链路”。

如果缺少一条清晰主线，学习 LLM 很容易陷入三个误区。第一，只记住很多名词，却不知道它们在工程链路里的位置。第二，把提示工程、RAG、微调、继续预训练混成一团，不知道何时该用哪一种。第三，过早沉迷单个专题，例如 LoRA 或 GraphRAG，却没有形成从“模型能力”到“业务交付”的整体视角。

## 1.3 核心概念

### 1.3.1 什么是工程视角下的大语言模型

大语言模型本质上是一个条件概率模型。给定上文  $x_{<t}$ ，它学习下一个 token 的条件分布  $p_{\theta}(x_t | x_{<t})$ 。从工程角度看，这意味着模型最原始的能力不是“理解世界”，而是“根据上下文继续生成最可能的序列”。

#### 定义 1.1 (工程视角下的 LLM)

在工程系统里，大语言模型可以被视为一个可调用的语义计算核心。它负责压缩上下文、完成推理、生成文本或结构化输出，但它本身不天然拥有业务真相、权限控制、外部状态和可追溯记忆。♣

### 1.3.2 主流模型家族与任务差异

从工程上看，LLM 并不是只有一种统一架构。至少可以把主流模型粗分为三类：

- **Encoder**：更擅长理解任务，例如分类、匹配、抽取和排序，典型代表是 BERT 一类模型。
- **Encoder-Decoder**：先完整理解输入，再生成输出，适合翻译、摘要、结构化改写等输入输出边界清晰的任务，典型代表是 T5。
- **Decoder-Only**：按自回归方式从左到右生成 token，天然适合开放式生成、对话与通用指令跟随，是当前大语言模型的主流路线。

对工程读者来说，最重要的不是死记这些名字，而是理解它们的偏好差异。若任务核心是“判断”和“抽取”，传统编码器模型在延迟与成本上往往更划算；若任务核心是“生成”和“多步语言交互”，Decoder-Only 模型通常更自然。企业知识助手之所以以大语言模型为主，是因为它同时需要问答、摘要、解释、澄清和多轮对话，而这些能力更贴近自回归生成范式。

### 1.3.3 训练目标决定模型的“天性”

模型的预训练目标会直接影响它后续最擅长的事情。最常见的两类目标是：

- **语言模型目标**：根据上文预测下一个 token。这会强化模型的续写、对话和生成能力。
- **去噪或掩码恢复目标**：恢复被遮盖、被打乱的输入片段。这更强调理解和重建能力。

这也是为什么不同模型在“问答像不像聊天助手”“摘要稳不稳”“抽取是否精确”这些维度上常常会呈现出不同风格。工程实践里，一个模型并不是越大越万能，而是会带着其训练目标留下的偏好进入下游场景。

### 1.3.4 为什么 Decoder-Only 会成为主流

过去几年里，Decoder-Only 架构之所以成为大语言模型的主流，不只是因为“它能聊天”，还因为它在大规模预训练和通用迁移上更符合当前行业路线：

- 它与开放式文本生成的实际使用方式一致，训练目标和部署目标更统一。
- 它可以直接利用海量无标注语料做自监督训练，不依赖任务标签。
- 在 zero-shot 和 instruction following 场景中，Decoder-Only 更容易把“语言继续生成”的能力迁移成“按要求回答”的能力。
- 它的工程生态更成熟，围绕推理加速、量化、适配器和部署的工具链也最完整。

这并不意味着 Encoder 或 Encoder-Decoder 没有价值，而是说明：如果一本书的主线是“构建企业知识助手”，那么围绕 Decoder-Only 组织知识最符合当代工程现实。

### 1.3.5 涌现能力与工程误读

旧稿里一个经常被提到的词是“涌现能力”。它通常指：当模型规模、数据规模和训练预算跨过某个阈值后，一些复杂能力会突然显得明显增强，例如多步推理、代码生成或复杂指令遵循。

但对工程读者来说，更重要的是避免对“涌现”做错误解读：

- **误读一**：把所有能力提升都归结为参数量增加。实际上，数据质量、训练目标和系统设计同样关键。

- **误读二：**把一次惊艳演示当成稳定能力。演示是点状现象，工程系统需要看分布、稳定性和回归表现。
- **误读三：**以为只要模型足够大，就不再需要检索、评测和权限系统。事实上，模型越强，越需要明确它该接管什么、不该接管什么。

### 1.3.6 三层能力边界

为了避免后续章节中概念混淆，本书先把能力边界拆成三层：

1. **模型能力：**阅读上下文、生成文本、遵循指令、完成通用推理。
2. **任务能力：**通过提示工程、SFT、PEFT、继续预训练或对齐训练后，在某类任务上表现更稳定。
3. **系统能力：**通过检索、工具调用、权限系统、日志与评测体系，把模型嵌入业务流程。



**笔记** 把“会回答企业文档里的问题”简单归因于模型本身，通常是错误的。很多时候，这种能力来自检索系统、文档解析、权限过滤和提示编排，而不是参数本体。

### 1.3.7 先判断问题属于哪一层

对初学者来说，最容易犯的错不是“不知道术语”，而是不知道当前问题属于哪一层。一个更稳妥的判断顺序是：

1. 如果答案依赖最新制度、权限边界或外部状态，先把它当成**系统能力问题**，优先考虑检索、工具和业务规则。
2. 如果模型知道事实，却总是格式不稳、语气不合规或任务边界模糊，再把它当成**任务能力问题**，考虑提示工程、SFT 或对齐。
3. 如果连通用阅读、摘要和指令遵循都表现很差，才优先怀疑**模型能力**或模型选择本身。

这套判断顺序看起来简单，却会直接决定项目成本。很多团队本可以先修检索，却误判成“模型太弱”；也有不少团队本该先统一输出格式，却一上来就做复杂训练。

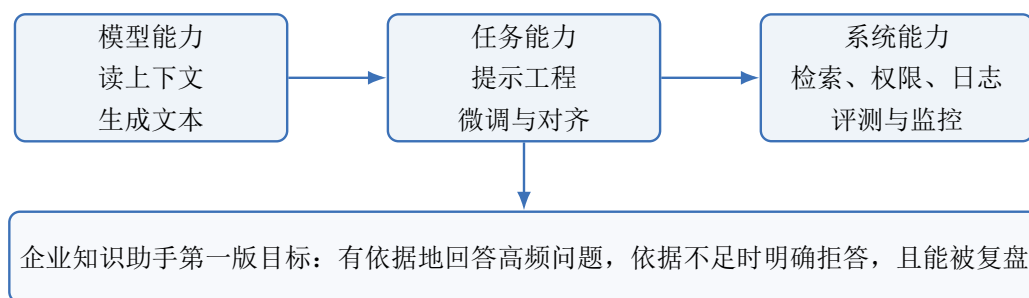


图 1.1: 企业知识助手的三层能力边界

### 1.3.8 为什么必须有一个贯穿案例

一本工程教材如果缺少持续推进的业务对象，知识就会碎片化。为了避免这一点，本书固定使用“企业知识助手”作为主案例。这个系统的目标是回答员工关于制度、产品、运维与项目流程的问题，并满足以下要求：

- 能处理多轮对话，并在当前会话中保持必要上下文。
- 对企业知识库中的答案给出可追溯依据，而不是只凭模型记忆作答。
- 在性能、成本与部署复杂度之间取得平衡。

- 能被评测、优化和持续迭代，而不是一次性演示系统。

## 1.4 主案例推进

我们把主案例拆成6个连续阶段，每一部分章节都对应推进其中一个阶段：

1. 先理解模型的语言生成机制，知道它擅长什么、不擅长什么。
2. 然后学会调用现成模型，完成最小可用的问答流程。
3. 接着通过提示工程、微调和继续预训练，让模型更贴近企业场景。
4. 再引入检索增强与文档理解，把企业知识接入系统。
5. 之后建立评测、对齐与性能优化机制，保证效果稳定。
6. 最后补齐推理部署和分布式训练视角，使系统可扩展、可维护。

从这一刻开始，读者应当把每章内容都看成是在回答一个具体问题：**企业知识助手要上线到可用状态，当前最需要补哪一块能力？**

为了让这条主线更清楚，本书默认企业知识助手的第一版目标不是“无所不能”，而是完成三个最小交付要求：

- 对高频制度和产品问题给出可读答案。
- 当依据不足时明确说“不知道”或“需要更多材料”。
- 能通过日志和评测知道系统为什么答对、为什么答错。

如果换成更工程化的表述，第一版并不追求“最聪明”，而是追求三个约束同时成立：

1. **可追溯**：答案最好能回到具体材料，而不是只凭参数记忆。
2. **可控**：面对权限、流程和高风险问题时，系统要知道何时拒答、何时转人工。
3. **可复盘**：回答好坏必须能被日志、评测和样本回放解释。

这三点看起来朴素，却几乎决定了后续所有技术选择。若一个团队把目标定成“做最强聊天机器人”，它会优先追求模型能力；若目标定成“做可上线的企业知识助手”，它就会更重视检索、评测、权限和部署稳定性。本书从一开始就选择后者。

## 1.5 常见坑与工程取舍

- **误把模型当数据库**：模型参数不是实时知识源，业务真相不能只靠预训练记忆。
- **误把 RAG 当万能药**：检索可以补知识，但不能自动解决推理链、权限控制和答案格式约束。
- **误把微调当前置步骤**：很多场景先用好提示词和检索，收益更高、成本更低。
- **误把单次演示当系统能力**：一次成功回答不代表系统具备稳定交付能力，必须引入评测与监控。

从旧稿反复出现的问题看，新手还有三类高频误区值得提前点出来：

- **把模型架构差异理解成“排行榜偏好”**：选型不是挑最火模型，而是看任务类型、成本预算和部署边界。
- **把生成能力等同于理解能力**：一个能写长答案的模型，不一定适合做高精度分类、抽取或排序。
- **把长上下文能力等同于“什么都能塞进去”**：上下文越长，越考验输入质量、检索质量和预算控制，而不是自动带来更好答案。



## 1.6 本章练习

 **练习 1.1 能力分层判断** 某团队的企业助手经常引用过期制度，但回答口吻和格式都很稳定。这个问题首先应归入哪一层能力问题？最优先的修复方向是什么？

**解** 这首先是**系统能力问题**，因为错误来自知识时效而不是表达风格。最优先的修复方向是补检索链路、更新知识源和建立依据引用，而不是先做微调。

 **练习 1.2 先判层再判手段** 某团队发现模型回答时常常漏掉“依据不足时请说明无法判断”这一规则，但给定材料通常是充分的。此时应先改什么？为什么不应直接上 RAG？

**解** 应先把问题当成**任务能力问题**，优先修改提示模板或补一小批 SFT 样本，强化“依据不足则拒答”的行为。因为当前材料通常充分，问题不在知识接入，而在模型没有稳定执行任务规则。

## 1.7 延伸阅读

- **Stephen Wolfram, *What Is ChatGPT Doing ... and Why Does It Work?***: 适合理解“下一个 token 预测”与语言行为之间的关系。
- **OpenAI / Anthropic / Google 的模型系统卡**: 适合理解能力边界、安全边界与部署边界是如何被写成工程约束的。
- **Chip Huyen, *AI Engineering***: 适合理解模型、数据与系统能力为何必须一起设计。

## 1.8 小结

本章建立了全书最重要的坐标系：大语言模型是语义计算核心，但不是完整系统；系统能力来自模型、数据、提示编排、检索和工程基础设施的协同；企业知识助手将作为贯穿案例，持续承载这些能力的落地过程。

## 1.9 下一章过渡

既然企业知识助手的底层仍是一个语言模型，那么下一步就必须回答：模型究竟如何“读懂上下文并继续生成”？下一章将聚焦 Transformer 的核心机制，用最少但足够的原理为后续所有工程章节打基础。

## 第 2 章 Transformer 核心机制：Attention、FFN、Norm 与损失

### 内容提要

- ❑ **本章核心问题：**模型为什么能够基于上下文继续生成，而企业知识助手中常见的“遗忘、跑偏、风格变化”等现象，又分别对应哪些 Transformer 核心机制？
- ❑ **主案例推进：**把企业知识助手中常见的错误症状，映射回 Attention、FFN、Norm 和训练目标这四个部件。
- ❑ **读完后能做什么：**看到长上下文答偏、训练不稳、格式漂移等现象时，先回到正确的底层机制解释它。

### 2.1 学习目标

- 理解 Transformer 中最关键的四个部件：Attention、FFN、Norm 和训练目标。
- 能从工程角度解释为什么这些机制会影响稳定性、速度与效果。
- 将核心机制映射回企业知识助手中的真实行为。

### 2.2 问题背景

很多工程问题表面上看是“调参问题”，本质上却来自对底层机制的误判。例如，为什么长上下文时模型更容易偏离主题？为什么换成 RMSNorm 后训练会更稳？为什么同样是激活函数，SwiGLU 往往比简单的 ReLU 更适合现代大模型？这些问题如果只靠经验记忆，很难迁移到新的模型和框架中。

因此，本章不追求完整推导，而是只保留工程读者必须掌握的最小机制集合，并把它们串成一条统一的数据流叙事：模型先看哪里，再决定如何变换表示，最后在什么目标下被训练。

### 2.3 核心概念

#### 2.3.1 Attention：模型如何在上下文中定位信息

自注意力的核心计算可以写成

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

这里的关键不是公式本身，而是它表达的行为：当前 token 会根据 query 与上下文 keys 的匹配度，对不同位置分配不同权重，再从 values 中聚合信息。

可以把它想成一种“上下文内检索”。例如，员工问“出差报销住宿上限是多少”，如果 prompt 前面堆了大量无关会议纪要，而真正的制度句子只出现在后面一小段，模型就必须在很长上下文里把注意力压到正确位置。压对了，答案就稳；压偏了，回答就会把无关背景也当成证据。

这里顺便要补上一个来自旧稿的重要背景：双向注意力和单向注意力服务的任务不同。双向注意力更像“阅读”，为了理解当前词，可以同时参考左右两边；单向注意力更像“写作”，预测当前 token

时只能看左边已经出现的内容。大语言模型之所以以因果、单向注意力为主，是因为它和开放式生成的使用方式最一致。

### 2.3.2 Attention 为什么会成为工程瓶颈

旧稿里多次强调，Attention 不仅决定模型“能看见什么”，还决定模型“要花多少钱才能看见”。当上下文长度增加时，注意力计算和 KV Cache 都会迅速变重，于是两个工程问题会同时出现：

- **训练侧**：显存占用增大，长上下文吞吐下降，训练更不稳定。
- **推理侧**：KV Cache 变大，解码时内存带宽压力上升，延迟更难压住。

这就是为什么后面讲长文本、RAG 和推理优化时，会反复回到 Attention。很多表面上像“部署问题”的症状，其实在这一层就已经埋下了根。

### 2.3.3 FFN 与激活函数：模型如何改写表示

Attention 解决“看哪里”的问题，前馈网络解决“看到之后如何变换表示”的问题。典型 FFN 可写成

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

激活函数  $\sigma$  决定了非线性表达能力。在现代大模型中，SwiGLU、GEGLU 之类门控激活常优于传统 ReLU，因为它们能在表达能力和训练稳定性之间取得更好平衡。

### 2.3.4 为什么现代大模型偏爱门控激活

如果只记住“LLaMA 用的是 SwiGLU”，却不知道为什么，那后面就很难读懂模型结构差异。门控激活的核心思想是：不仅做非线性变换，还让一条额外分支决定哪些信息应该被保留、哪些应该被抑制。

以 SwiGLU 为例，可以近似写成

$$\text{SwiGLU}(x) = \text{Swish}(xW) \otimes (xV),$$

其中  $\otimes$  表示逐元素乘法。它比标准 ReLU FFN 更像“带阀门的表示变换器”。工程上通常有三点收益：

- 表达能力更强，尤其在格式变换和复杂语言模式上更稳定。
- 梯度更平滑，训练时不容易出现过于粗糙的截断。
- 在相近参数预算下，经常能换到更好的任务效果。

旧稿中还提到一个常被忽略的工程细节：当采用 GLU 类结构后，FFN 的中间维度常常需要重新设计，以保持整体参数量与吞吐的平衡。因此“不同模型 FFN 为什么宽度不同”并不只是风格问题，很多时候是在给激活函数结构让路。

### 2.3.5 Norm：模型如何保持训练稳定

LayerNorm 的常见形式为

$$\text{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

它的作用不是“让数值更漂亮”，而是控制不同层之间的表示尺度，使深层网络在反向传播时更稳定。RMSNorm 则进一步去掉均值中心化，只保留均方根缩放，计算更简洁，常见于现代解码器模型。



**笔记** 当你在训练日志里看到 loss 振荡、梯度异常或不同层更新不均匀时，Norm 设计通常是需要优先排查的对象，而不是最后才关注的细节。

### 2.3.6 RMSNorm、DeepNorm 与位置设计

围绕 Norm，旧稿里至少有三个值得保留的工程结论。

**第一，RMSNorm。**它在很多现代大模型里替代了 LayerNorm，因为它更简洁、开销更低，又能保留足够的稳定性。这说明现代模型并不是一味追求“更复杂”，而是在效果和效率之间反复折中。

**第二，DeepNorm。**它的目标是让更深的模型在训练中保持稳定更新。对工程读者来说，不需要死记具体初始化公式，只要知道：**模型越深，残差路径和归一化尺度就越决定训练成败。**

**第三，Norm 放在什么位置。**旧稿中专门区分了 Post-LN 和 Pre-LN。工程上可以把它概括成一句话：深层大模型更偏爱 Pre-LN，因为它通常更稳定；如果看到一个模型训练很容易炸，而另一个相近模型却很稳，Norm 位置常常就是关键差异之一。

### 2.3.7 交叉熵与 KL：模型训练到底在优化什么

自回归预训练最常见的目标是最小化交叉熵损失：

$$\mathcal{L}_{\text{CE}} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}).$$

它衡量模型对标准答案 token 的预测是否足够集中。

KL 散度则用于衡量两个分布的差异：

$$D_{\text{KL}}(p||q) = \sum_i p(i) \log \frac{p(i)}{q(i)}.$$

在对齐训练、蒸馏和策略优化中，KL 常被用来约束新模型不要偏离参考模型过远。

### 2.3.8 为什么分类、生成和对齐都会绕回这些损失

对工程实践而言，交叉熵和 KL 不是只会在公式区出现的名词，而是训练现象的解释器：

- 在预训练里，交叉熵决定模型是否更像训练语料那样继续生成。
- 在 SFT 里，交叉熵决定模型是否更像示范答案那样组织输出。
- 在对齐训练里，KL 决定新策略能离原始语言能力偏多远。

旧稿里还谈到了 Softmax 的数值稳定性、多任务 loss 失衡和为什么分类通常不用 MSE。这些内容都指向同一个结论：**loss 不是一个抽象分数，而是优化器真正追逐的目标。**如果目标定义错了，训练越稳定，偏得越远。

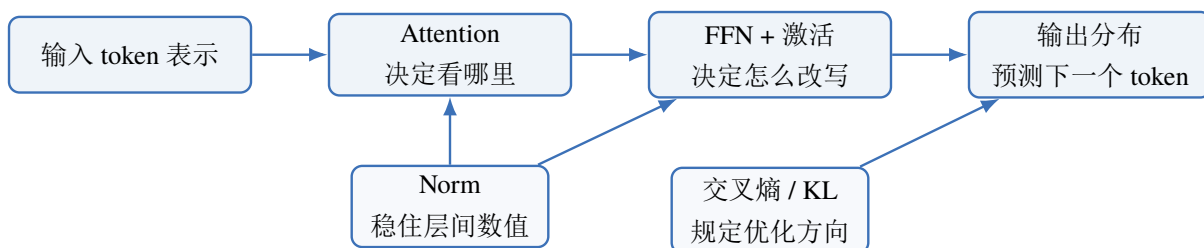


图 2.1: Transformer 单层的工程化理解：看哪里、怎么变换、朝哪优化

### 2.3.9 现代 Attention 优化：MQA、GQA 与 FlashAttention

旧稿里关于 Attention 的一个重要延伸，是现代模型为了解决长上下文与推理成本而做的结构优化。

**Multi-Query Attention (MQA)** 的核心思路是让多个 query 头共享 key 和 value，这样能显著缩小 KV Cache，推理时显存更省、速度更快。

**Grouped-Query Attention (GQA)** 是 MQA 和标准多头注意力之间的折中：不是所有头都共享同一组 key/value，而是按组共享。这样通常能在效果与推理成本之间取得更平衡的结果。

**FlashAttention** 则更像硬件感知优化。它不改变 Attention 的任务定义，但通过分块计算和更高效的内存访问，大幅减少高带宽显存的搬运压力。工程上看到“更快”“更省显存”时，很多时候并不是模型学会了更少的东西，而是这些底层实现被改进了。

### 2.3.10 并行 Transformer Block 与结构折中

旧稿还提到一种值得记住的趋势：为了训练吞吐和部署效率，一些模型会把原本更串行的子模块改成更并行的结构。它提醒我们，Transformer 的演化不只是围绕“更强表示能力”，也围绕“更符合硬件现实”。后面讲推理与分布式训练时，这种结构优化会再次出现。

### 2.3.11 从症状反推机制

工程学习最怕“学完原理，遇到问题还是不会判断”。下面给出一组最常见的现象到机制的映射：

- **长上下文时更容易答偏**：优先回看 Attention、上下文长度和无关信息比例。
- **格式经常漂移**：优先看训练目标、提示模板和任务数据，而不是只怀疑解码参数。
- **微调过程不稳定**：优先看 Norm、学习率、批次大小和梯度行为。
- **训练 loss 降了，业务效果没涨**：优先看交叉熵目标和真实任务目标之间是否脱节。

再补一组更偏工程侧的症状映射：

- **显存突然吃紧**：除了模型参数量，也要优先怀疑 Attention 结构和 KV Cache。
- **更换模型后训练稳定性差异很大**：优先看 Norm 类型、Pre-LN/Post-LN 与初始化策略。
- **同样参数量下新模型推理更快**：很可能是 MQA、GQA、FlashAttention 或 Kernel 优化在起作用。

## 2.4 主案例推进

回到企业知识助手，这些机制分别解释了系统中的关键现象：

- 员工提问中一旦混入大量无关背景，Attention 就可能把权重分散到错误片段上，导致回答跑偏。
- 当我们要求模型输出固定格式、摘要或工具调用参数时，FFN 与激活函数承载了大量“格式变换”的能力。
- 如果后续对模型进行微调或继续预训练，Norm 设计会直接影响训练稳定性与收敛速度。
- 当我们谈论“模型更像企业内部写作风格了”，本质上就是交叉熵目标在新数据分布上被重新优化了。

理解这些机制的意义，在于后续不再把所有现象都归结为“模型太笨”或“数据不够多”，而是能定位到是上下文建模、表示变换、训练稳定性还是目标函数的问题。

进一步说，企业知识助手后续会遇到的很多真实工程问题，都已经在本章埋下了答案：



- 为什么检索片段一多，回答反而更差？本质和 Attention 的选择压力有关。
- 为什么换一个模型后显存和吞吐差别明显？往往和 MQA、GQA、FlashAttention 这类结构优化有关。
- 为什么同样是微调，有的模型很稳，有的模型频繁 loss spike？Norm 与初始化设计常常是隐藏变量。

## 2.5 常见坑与工程取舍

- 只关注参数量，不关注上下文机制：更大的模型不一定能更好处理长文档，Attention 设计同样关键。
  - 把激活函数当次要实现细节：在同等算力预算下，激活函数会影响吞吐、收敛和最终效果。
  - 忽略损失函数和业务目标的差距：交叉熵最优化的是“像标准答案”，不等于“对用户最有帮助”。
  - 误用 KL 约束：KL 太强会让新策略几乎学不到东西，太弱又会导致模型行为漂移。
- 沿着旧稿里的经验总结，还可以再提醒三类误区：
- 看到长上下文问题就盲目追求更长窗口：如果输入质量差，窗口更长只会把噪声放大。
  - 把所有推理性能问题都归因于框架：很多瓶颈其实来自 Attention 结构本身。
  - 把 loss 当成唯一训练真相：loss 更低，不一定意味着企业任务更可用。

## 2.6 本章练习

 **练习 2.1 长上下文答偏** 企业知识助手在长制度文档问答中，明明相关句子已经被放进 prompt，却总是引用前文无关会议纪要。这个现象更可能首先对应哪个机制？应优先缩短什么内容？

**解** 更可能首先对应 Attention 的问题。模型在过长上下文中把注意力分散到了无关片段，应优先缩短无关历史消息和低相关检索片段，而不是一开始就怀疑模型完全不懂制度内容。

 **练习 2.2 损失与业务目标** 某次 SFT 后训练 loss 明显下降，但模型仍经常不给依据，只直接输出结论。这个现象说明了什么？

**解** 这说明训练目标与业务目标之间可能存在脱节。交叉熵下降只说明模型更像训练答案，不说明它一定学会了“先给依据再给结论”。应先检查训练样本是否真的把依据写成必选输出结构，而不是只看 loss。

## 2.7 延伸阅读

- Vaswani et al., *Attention Is All You Need*: 理解 Transformer 的最初设计意图。
- Llama、Qwen、Mistral 等模型技术报告：适合理解现代解码器在 Norm、激活和长上下文设计上的实际取舍。
- Sebastian Raschka, *Build a Large Language Model (From Scratch)*: 适合把公式和实现细节对应起来。

## 2.8 小结

本章给出了工程上最值得掌握的四个底层机制。**Attention** 决定上下文选择，**FFN** 与激活函数决定非线性表达，**Norm** 决定训练稳定性，交叉熵与 **KL** 决定模型最终会朝哪个方向优化。

## 2.9 下一章过渡

理解底层机制后，下一步就应该把它们接到真实代码与推理链路中。下一章将从 **Hugging Face Transformers** 出发，建立从“加载模型”到“读输出、调采样、做调试”的最小使用闭环。

## 第3章 从库到模型：Transformers 使用、推理与调试基础

### 内容提要

- ❑ 本章核心问题：如何把一个现成大语言模型稳定接入代码，形成一条可调试、可记录、可扩展的最小问答链路？
- ❑ 主案例推进：为企业知识助手搭起第一条“输入问题到输出答案”的可复盘推理链。
- ❑ 读完后能做什么：能区分 token 化、前向计算、logits、采样和后处理各自负责什么，输出失控时知道先看哪一段。

### 3.1 学习目标

- 掌握使用 Transformers 加载分词器和因果语言模型的基本流程。
- 理解输入张量、logits、解码参数和输出结构在工程里的意义。
- 能为企业知识助手搭建最小可用的模型调用链路。

### 3.2 问题背景

很多初学者第一次接触大模型工程时，往往直接从一个封装好的聊天界面开始，结果后续遇到性能问题、输出格式问题或 token 截断问题时，不知道应该在模型层还是应用层排查。工程学习需要从更底的一层开始：明确模型是怎样被加载、编码、生成和解析的。

对于教材读者来说，真正关键的不是“会跑通一段 demo”，而是把推理链路拆开看：问题如何变成 token，token 如何变成 logits，logits 如何在采样后变成最终文本。

### 3.3 核心概念

#### 3.3.1 最小模型调用链路

一个最小的因果语言模型调用链路通常包含五步：

1. 加载 tokenizer，定义文本如何映射为 token。
2. 加载模型权重，明确运行精度与设备。
3. 把输入编码成张量，送入模型做前向计算。
4. 根据 logits 做采样或贪心选择，得到输出 token。
5. 对输出 token 进行解码，并结合业务规则做后处理。

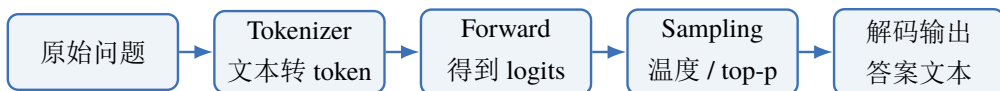


图 3.1: 从文本输入到输出答案的最小推理链路



### 3.3.2 先跑通，再加复杂度

对初学者来说，最稳妥的顺序不是一开始就接聊天模板、多轮记忆和工具调用，而是先把单轮、单模型、单请求跑通。只有最小链路足够透明，后续任何增强模块出问题，团队才知道应该回退到哪一层排查。

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "Qwen/Qwen2.5-1.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

prompt = "请用三句话说明什么是检索增强生成。"
inputs = tokenizer(prompt, return_tensors="pt")
outputs = model.generate(
    **inputs,
    max_new_tokens=128,
    temperature=0.3,
    top_p=0.9
)
text = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

### 3.3.3 logits 与采样参数为什么重要

`generate()` 并不是黑盒魔法。模型先输出每个位置上“下一个 token 的概率倾向”，也就是 logits；随后解码策略再决定如何从这个分布里取样。

`max_new_tokens` 控制生成长度上限，`temperature` 控制分布平滑程度，`top_p` 控制采样时保留的累计概率质量。它们共同决定回答是更稳定还是更多样。对企业知识助手这类强调稳定性的系统，一般优先采用低温度、适中的 `top_p` 和明确的最大输出长度。

### 3.3.4 为什么要读懂模型输出结构

工程里不能只拿最终文本，还要知道模型在中间阶段产生了什么。例如：

- token 长度是否超过预算。
- 是否提前遇到结束符。
- 某次输出是否因为截断或模板错误而偏离预期。
- 是否需要保存 logits、attention 或 hidden states 做进一步分析。

### 3.3.5 调试大模型时先看哪里

最有效的调试顺序通常是：

1. 先看 prompt 是否真的按预期拼接。
2. 再看输入 token 长度与截断策略。
3. 然后看采样参数是否过于激进。

#### 4. 最后才怀疑模型权重本身存在问题。

这个顺序看似保守，其实是在避免一种常见误诊：把工程实现问题误判成模型能力问题。很多所谓“模型忽然变差”，最后都能追溯到 **prompt** 改坏了、输入被截断了，或默认采样参数被悄悄换掉了。

### 3.4 主案例推进

在企业知识助手的第一版里，我们先不引入检索、记忆和工具调用，只要求系统能接受一个问题并生成可读答案。这个阶段的目标不是“回答完全正确”，而是建立一条可验证的最小闭环。

第一版系统应该记录以下信息：

- 原始输入问题。
- 实际送入模型的完整 **prompt**。
- 输入 **token** 数和输出 **token** 数。
- 生成参数配置。
- 模型返回文本与耗时。

下面是一份足够支撑第一版排查的最小日志结构：

```
trace = {
  "question": question,
  "prompt": prompt,
  "input_tokens": len(inputs["input_ids"][0]),
  "generation_config": {
    "max_new_tokens": 128,
    "temperature": 0.3,
    "top_p": 0.9
  },
  "output_text": text,
  "latency_ms": latency_ms
}
```

这类日志最大的价值不在“记录得更全”，而在于它让团队第一次拥有了复盘样本的能力。没有复盘，后续所有优化都只能停留在主观印象层面。

### 3.5 常见坑与工程取舍

- 只保存最终文本，不保存 **prompt**：后续几乎无法复盘失败样本。
- 默认采样参数不加区分：创作型参数直接拿来做法知识问答，容易让输出漂移。
- 忽略 **tokenizer** 差异：不同模型的聊天模板、特殊 **token** 和截断规则不完全一致。
- 把业务错误误判为模型错误：很多失败是输入拼接、模板拼装或后处理造成的。

### 3.6 本章练习

 **练习 3.1 输出失控先看哪一层** 某次版本更新后，企业知识助手开始输出很长的发散回答，但模型权重没有变化。你应先检查哪两类信息？

**解** 应先检查 prompt 是否被改坏以及采样参数是否被调得更激进。模型权重没变时，最常见的原因是输入拼接和解码配置发生了变化，而不是模型突然失去能力。

 **练习 3.2 复盘最小日志** 如果日志里只能保留三项字段来支持后续复盘，你会优先保留哪三项？为什么？

**解** 优先保留完整 prompt、输入输出 token 数、生成参数配置。完整 prompt 决定模型实际看到了什么，token 数决定是否存在截断和预算问题，生成参数决定输出是否因为采样策略而漂移。这三项能最快把问题缩到正确层面。

## 3.7 延伸阅读

- **Hugging Face Transformers 官方文档**：最适合把 API 调用与模型行为逐项对应起来。
- **vLLM、TGI 等推理服务文档**：适合理解从本地单请求到服务化推理之间多了哪些工程层。
- **OpenAI / Anthropic 的 Prompting 指南**：适合对照理解“模型调用 API”与“输入编排”之间的关系。

## 3.8 小结

本章把大模型从“抽象概念”落实为可执行链路：加载、编码、前向、采样、解码和日志记录。对工程实践而言，能读懂这条链路，比会调几个孤立参数更重要。

## 3.9 下一章过渡

当最小问答链路跑通之后，新的问题马上出现：同一个模型为什么有时回答稳定，有时又前后不一致？下一章将进入提示工程与上下文管理，讨论如何通过输入设计让模型行为更可控。

## 第 4 章 提示工程与上下文管理：Prompting、采样与长期记忆

### 内容提要

- ❑ **本章核心问题：**在不改模型参数的前提下，应该如何组织系统指令、历史消息和知识上下文，才能让企业知识助手回答得更稳定、更可控？
- ❑ **主案例推进：**把企业知识助手的输入编排拆成系统层、会话层、知识层和用户层四层结构。
- ❑ **读完后能做什么：**知道什么时候继续改 prompt，什么时候该停止提示词微调，转去修检索、数据或训练。

### 4.1 学习目标

- 明确提示工程的职责边界，知道它能解决什么、不能解决什么。
- 掌握上下文窗口、消息模板、会话记忆和长期记忆的基本设计思路。
- 为企业知识助手建立可演进的输入编排方案。

### 4.2 问题背景

在很多真实项目里，团队最早接触到的不是微调，而是提示工程。原因很现实：改 prompt 的成本最低，反馈最快，且通常不需要训练资源。然而，提示工程最容易被神化。它确实能显著改善模型输出，但无法替代知识检索、数据建设和系统约束。

因此，我们需要把提示工程放回正确位置：它是**输入编排层**，负责让模型在给定能力边界内尽可能稳定地工作。

### 4.3 核心概念

#### 4.3.1 提示工程的三项职责

从工程视角看，提示工程主要负责三件事：

1. **定义角色：**告诉模型它应该扮演什么身份，输出什么风格。
2. **约束任务：**明确输入、输出、禁止项和评分标准。
3. **组织上下文：**把系统指令、用户问题、检索片段和历史对话拼成一条高质量上下文。

#### 4.3.2 上下文窗口不是越长越好

上下文窗口越大，可容纳信息越多，但工程上并不意味着“一次塞满所有信息”。因为：

- 长上下文会增加显存和延迟成本。
- 无关信息会稀释注意力，影响答案聚焦。
- 历史消息越多，模板越难维护，错误越难排查。

好的上下文管理不是“尽量多放内容”，而是“只放当前答案真正需要的信息”。

### 4.3.3 短期记忆与长期记忆

在多层系统里，我们可以把记忆拆成两类：

- **短期记忆**：当前会话中的任务状态、用户意图和最近若干轮上下文。
- **长期记忆**：跨会话保留的用户偏好、历史事实和可复用背景。

短期记忆更像 prompt 的一部分，长期记忆更像被结构化管理的数据库资产。二者不能混在一层处理。

### 4.3.4 什么时候该继续改 prompt，什么时候不该

提示工程很有用，但它不是默认答案。更稳妥的判断方式是：

- 如果问题是角色不清、格式不稳、拒答规则不明确，优先继续改 prompt。
- 如果问题是知识缺失、事实过期或权限边界不清，优先补检索、工具或业务规则。
- 如果问题在同类样本上反复出现，且 prompt 已经很清楚，再考虑 SFT 或其他训练手段。

### 4.3.5 一个可维护的消息模板

```
messages = [  
    {"role": "system", "content": "你是企业知识助手，优先依据提供的材料回答。"},  
    {"role": "system", "content": "如果材料不足，请明确说明依据不足。"},  
    {"role": "user", "content": question}  
]
```

模板的关键不是“写得多复杂”，而是职责清楚：哪些指令长期稳定，哪些内容来自检索，哪些内容是当前问题，必须在代码层分开管理。

### 4.3.6 上下文裁剪顺序

当上下文预算不够时，裁剪顺序同样需要设计。对企业知识助手来说，更合理的顺序通常是：

1. 保留系统层规则，不轻易裁掉。
2. 保留当前问题和本轮任务约束。
3. 优先保留最相关的检索片段，而不是机械保留全部历史对话。
4. 只保留仍然影响当前回答的近几年会话。
5. 长期记忆只保留少量确定有用的结构化字段。

## 4.4 主案例推进

在主案例中，我们把输入编排分成四层：

1. **系统层**：回答风格、保守原则、引用要求。
2. **会话层**：当前轮之前必须保留的任务上下文。
3. **知识层**：后续由 RAG 提供的文档片段与元数据。
4. **用户层**：本轮问题与格式要求。

在只有基础模型的阶段，我们先实现前两层。等到 RAG 模块上线，再把知识层插入进去。这样做的好处是，提示模板不会因为系统演进而频繁推翻重写。

下面给出一份适合企业知识助手初版使用的“上下文预算清单”：

| 上下文层 | 建议保留内容          | 裁剪原则             |
|------|-----------------|------------------|
| 系统层  | 身份、拒答规则、引用规则    | 最后裁剪，尽量长期稳定      |
| 会话层  | 当前任务状态、最近几轮澄清   | 只保留仍影响当前回答的轮次    |
| 知识层  | 高相关检索片段、文档标题、来源 | 优先保留高相关且权威的片段    |
| 用户层  | 当前问题、格式要求       | 必须保留，不能因为压缩而失真   |
| 长期记忆 | 明确用户偏好、角色信息     | 只保留可解释、可删除的结构化字段 |

再看一个连续提示词迭代的小例子。假设第一版系统提示只有“回答员工问题”，那么模型很容易自由发挥。第二版改成“优先依据材料回答，若材料不足请明确说明”，系统就已经开始具备保守边界。第三版再加入“先给结论，再给依据，再给建议动作”，输出结构才真正稳定。这个过程说明，提示工程的价值不是“神奇措辞”，而是逐步把系统规则显式化。

## 4.5 常见坑与工程取舍

- 把所有规则都写进一个超长 prompt：可读性差，维护成本高，也不利于定位错误。
- 记忆与知识不分层：用户偏好、历史任务和文档事实混在一起，会让系统状态失控。
- 依赖单轮提示词技巧解决结构性问题：当事实缺失或知识过期时，单靠 prompt 无法补救。
- 过度追求“拟人化长期记忆”：很多业务场景需要的是准确、可控和可追踪，而不是情感陪伴式记忆。

## 4.6 本章练习

 **练习 4.1 提示修复** 企业知识助手明明拿到了材料，但经常先给一段口语化结论，最后才勉强补一句“依据见上文”。如果只能先改 prompt，不改模型参数，你会优先补哪一条系统规则？

**解** 应优先补输出顺序规则，例如“先给结论，再给依据，再给建议动作；如果材料不足，明确说明无法判断”。这类问题首先是任务约束不够清楚，而不是知识缺失。

 **练习 4.2 记忆设计** 为什么企业知识助手不应在早期就采用“自动记住一切”的长期记忆方案？

**解** 因为早期系统最需要的是准确、可控、可删除。自动记住一切会把用户偏好、过时事实和无关会话混成一层，既难解释，也难清理，还会推高上下文成本。更合理的做法是只保留少量结构化且确定有用的长期字段。

## 4.7 延伸阅读

- *Anthropic, Prompt Engineering Interactive Tutorial*：适理解系统提示、示例与约束的分工。

- **OpenAI Prompting 指南**：适理解“指令清晰化”在工程中的基本范式。
- **LangChain / LlamaIndex 的 Memory 与 Prompt 模块文档**：适理解输入编排怎样落到框架接口上。

## 4.8 小结

提示工程的价值在于把模型输入组织得更清晰、更稳定、更贴近业务目标。它解决的是“如何让已有能力发挥出来”的问题，而不是“如何凭空创造新能力”的问题。

## 4.9 下一章过渡

当提示工程已经做到位，但模型仍然不够贴合领域风格或任务边界时，就需要进一步改造模型本身。下一章将进入微调路线图，讨论何时使用 SFT、PEFT、LoRA 与 QLoRA。



---

## 第二部分

# 训练与改造模型

## 第 5 章 微调路线图：SFT、PEFT、LoRA、QLoRA 与 PEFT 实战

### 内容提要

- ❑ 本章核心问题：当提示工程已经不够时，企业知识助手究竟应该选择 SFT、PEFT、LoRA 还是 QLoRA，选择依据是什么？
- ❑ 主案例推进：把企业知识助手从“会用模型”推进到“会改模型”，建立一条成本可控的改造路线。
- ❑ 读完后能做什么：面对真实问题时，不再把“微调”当统一答案，而能先判定是改提示、做 SFT、上 LoRA，还是继续预训练。

### 5.1 学习目标

- 理解提示工程、SFT、PEFT 和继续预训练在能力改造中的分工。
- 掌握 LoRA 与 QLoRA 的基本思想、适用场景与工程代价。
- 能为企业知识助手设计一条成本可控的模型改造路线。

### 5.2 问题背景

当企业知识助手的基础问答链路已经跑通时，团队通常会遇到两个新问题。第一，模型“听得懂”，但说话风格不符合企业要求，例如回答过于口语化、格式不稳定或缺少依据说明。第二，模型在某些垂直任务上仍然不够可靠，例如内部工单分类、术语解释或标准流程问答。

这时常见的误区是直接进入“大模型训练”模式，仿佛只有重新训练权重才算真正优化模型。事实上，大多数工程团队首先需要的是一条**分层改造路线**：先判断问题能否通过提示工程解决，再判断是否需要监督微调，最后才决定是否进一步采用参数高效微调或更重的训练方案。

### 5.3 核心概念

#### 5.3.1 模型改造的四层路线

我们把常见改造手段按成本与侵入性从低到高排成一条路线：

1. **提示工程**：不改参数，只改输入编排。
2. **监督微调（SFT）**：用高质量指令数据直接优化模型输出风格和任务行为。
3. **参数高效微调（PEFT）**：只训练少量新增参数，以较低成本适配任务。
4. **继续预训练或更大规模训练**：当领域分布差异极大时，再考虑更重的训练方式。

### 5.3.2 SFT：让模型学会“该怎样回答”

SFT 的目标仍然是最小化交叉熵损失，只是训练数据从通用语料换成了更贴近任务的指令样本：

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log p_{\theta}(y_t | x, y_{<t}).$$

这里  $x$  是指令输入， $y$  是理想输出。SFT 的作用通常包括：

- 统一回答风格与格式。
- 增强任务遵循能力。
- 让模型更稳定地处理某类常见输入。

### 5.3.3 PEFT、LoRA 与 QLoRA

全参微调效果往往不错，但成本高、存储重、复用差。PEFT 的核心思路是：冻结原始大模型的大部分参数，只训练少量新增模块或低秩参数，从而在计算、显存和部署上更友好。

LoRA 的核心思想是把权重增量  $\Delta W$  低秩分解为

$$\Delta W = BA,$$

其中  $A$  和  $B$  的秩远低于原矩阵维度。这样可以显著减少可训练参数量。

QLoRA 则进一步把基座模型量化到较低精度，同时训练 LoRA 适配器，从而把单卡可微调的模型规模进一步推大。它的工程意义很直接：在资源有限时，把“能不能训”这个问题先变成“训得慢一点但可以训”。

### 5.3.4 PEFT 家族不是只有 LoRA

旧稿中专门有一整章在讲 PEFT，不是为了堆名词，而是因为“只改一小部分参数”本身就有很多实现路线。对工程读者来说，至少要知道下面这几类思路：

- **Adapter-Tuning**：在原模型层间插入小模块，只训练这些新增层。
- **Prompt-Tuning / Prefix-Tuning / P-Tuning**：不直接改主体权重，而是学习一组连续 prompt 或前缀表示。
- **LoRA 系列**：不插新层，而是对原权重的增量做低秩分解。

它们的共同点是降低训练成本，不同点在于：改动模型结构的程度不同、推理时额外开销不同、对任务类型的适配能力也不同。新版全书以 LoRA/QLoRA 为主讲，是因为它们在当前开源工程里最常见，但这并不意味着其他 PEFT 路线没有价值。

### 5.3.5 为什么 LoRA 会成为工程默认项

LoRA 在工程上流行，不是因为它“最先进”，而是因为它在几个关键维度上很均衡：

- **训练成本低**：只训练低秩矩阵，显存压力远小于全参微调。
- **部署灵活**：可以单独保存适配器，按场景切换，而不必复制整套模型权重。
- **生态成熟**：Hugging Face PEFT、各类训练脚本和服务框架都已经把它当成默认支持项。

从企业知识助手视角看，这一点尤其重要。因为一个企业内部往往不是只有一个固定场景，而是会同时出现制度问答、工单摘要、流程解释、术语说明等多个子任务。如果每个任务都维护一套全量微调权重，运维成本会很快失控；但如果每个任务只维护一套适配器，系统就容易得多。

### 5.3.6 QLoRA 解决的不是“效果问题”，而是“可训练性问题”

很多初学者会把 QLoRA 理解成“LoRA 的升级版”，这不够准确。QLoRA 首先解决的是资源受限时根本训不起来的问题。它通过对基座模型做低比特量化，把显存占用压下来，再在量化权重之上训练 LoRA 适配器。

因此，QLoRA 最适合的不是“任何都想上量化的场景”，而是下面这种局面：

- 你已经判断这个任务需要训练，而不是只改 prompt。
- 你手里的 GPU 不足以舒服地做全参微调。
- 你希望在可接受的精度损失范围内，把实验先跑起来。

这也是为什么在很多真实项目里，QLoRA 的价值不在于“绝对最好”，而在于它让团队从“完全做不了训练”进入“可以低成本迭代训练”的状态。

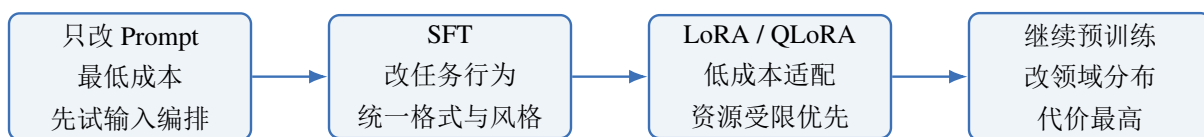


图 5.1: 企业知识助手的模型改造决策阶梯

### 5.3.7 先做选择，再做训练

在真正启动训练之前，团队最好先回答四个判断问题：

1. 问题主要是知识不在模型里，还是模型回答方式不对？
2. 需求是追求最新事实，还是追求稳定风格和任务行为？
3. 线上部署是单一场景，还是要在多个业务场景之间切换？
4. 当前算力预算是能做全参微调，还是只能接受适配器方案？

如果把这四个问题再压成一条更实用的决策链，可以得到下面的判断顺序：

1. 先问事实是否动态：如果答案依赖频繁更新的知识，优先考虑 RAG，而不是训练。
2. 再问输出是否不稳：如果问题主要是格式、风格和任务边界，优先考虑 SFT。
3. 再问预算是否够：如果要训练但预算有限，优先 LoRA/QLoRA。
4. 最后才问是否要改语言分布：只有领域文本本身很陌生，才进入继续预训练。

这条链的价值在于，它能阻止团队过早把所有问题都升级成“训练问题”。

### 5.3.8 一个最小 SFT 样本

对工程读者来说，理解训练样本往往比背诵概念更重要。一个最小的企业问答 SFT 样本可以是：

```

sample = {
  "instruction": "根据给定制度回答员工问题。",
  "input": "问题：出差报销住宿上限是多少？\n材料：...",
  "output": "结论：...\n依据：...\n如需例外审批，请联系..."
}
  
```

它传达了一个核心原则：SFT 训练的并不是“企业里有哪些事实”，而是“模型拿到这类输入时，应该按什么方式回答”。

## 5.4 主案例推进

对于企业知识助手，我们采用一条保守但高收益的路线：

1. 先用提示工程统一回答格式，例如“先给结论，再给依据，再给建议动作”。
2. 如果格式仍不稳定，或模型对企业术语理解明显不足，再建设一批高质量 SFT 数据。
3. 资源有限时，优先采用 LoRA 或 QLoRA 做任务适配，而不是一上来全参微调。
4. 只有当领域文本与基础模型分布差异很大，例如大量内部缩略语、制度文本和行业术语混杂时，才考虑继续预训练。

这一路线的关键是：微调不是为了把一切问题都塞进模型参数，而是为了让模型在稳定性和任务习惯上更像一个合格的企业助手。

在旧稿中，还有几类非常典型的微调场景判断，值得在这里直接总结成规则：

- 回答风格混乱：先做提示工程，再考虑小规模 SFT。
- 任务输出格式不稳：优先做 SFT 或 LoRA，而不是继续预训练。
- 模型读不懂内部术语或日志格式：优先怀疑领域语言分布问题，考虑继续预训练。
- 多场景并存且频繁切换：优先适配器路线，而不是多套全量权重。

如果把企业知识助手拆成更细的子任务，可以得到一张更贴近实际的路线图：

1. 对“标准制度问答”先靠 prompt + RAG 跑通。
2. 对“固定格式工单摘要”补 SFT。
3. 对“多个部门风格不同的输出模板”考虑 LoRA 适配器。
4. 对“内部缩略语和术语密集文本”再考虑继续预训练。

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)
```

旧稿里关于 PEFT 库实操的材料，真正重要的不是 API 名字，而是下面几个工程习惯：

- 明确 target\_modules，不要想当然地把所有线性层都挂上 LoRA。
- 训练前先确认可训练参数比例，避免“以为在做 PEFT，实际几乎改了全模型”。
- 适配器单独保存、单独命名，建立任务到适配器版本的映射关系。
- 推理时明确是“合并权重”还是“按需加载适配器”，因为这会影响服务架构设计。

### 5.4.1 LoRA 训练中最容易踩的坑

结合旧稿里的经验，LoRA/QLoRA 最常见的风险不是“概念不会”，而是工程判断失误：

- rank 并非越大越好：可训练参数变多不一定线性带来收益，反而会增加过拟合和显存负担。

- 盲目挂载模块：如果目标任务只是回答风格微调，不一定需要给所有注意力投影层都加适配器。
- 忽略训练数据质量：LoRA 再轻，也挡不住坏数据把模型往坏方向推。
- 只看 loss，不做样本回放：很多 LoRA 训练会在 loss 看起来正常时，悄悄把输出变得模板化或空洞。

## 5.5 常见坑与工程取舍

- 把知识缺失误判为微调问题：如果答案依赖频繁变化的企业文档，优先考虑 RAG，而不是把知识写死进参数。
- 只看训练 loss，不看真实任务样本：SFT 收敛并不意味着回答更有帮助。
- LoRA 模块选得过多：适配器越大，收益未必线性增长，但显存和训练复杂度会明显上升。
- 忽略推理部署形态：如果线上要同时切换多个企业场景，适配器方案通常比维护多套全量权重更现实。

再补三条旧稿里反复出现、但很容易在新书里被压掉的提醒：

- 灾难性遗忘不是全参微调专属问题：适配器方案也可能因为训练数据过窄而把模型行为推向过度模板化。
- 微调前先做评测集：否则训练完只能靠印象判断“是不是更好了”。
- 不要把 LoRA 当成低成本借口：成本低不等于可以跳过数据清洗、评测和部署设计。

## 5.6 本章练习

🔥 **练习 5.1 先改提示还是上 LoRA** 企业知识助手已经能答对大部分问题，但经常不按“结论-依据-建议动作”的固定格式输出。知识来源本身没有问题。此时你应先做什么？

解 应先改提示模板，必要时再补一小批 SFT 样本，而不是直接上 LoRA。因为当前问题主要是回答方式不稳定，不是知识缺失，也不是必须通过参数适配才能解决的能力缺口。

🔥 **练习 5.2 什么时候上 QLoRA** 在什么情况下，QLoRA 比全参微调更适合作为企业知识助手的第一种训练方案？

解 当团队算力和显存预算有限、目标是快速让模型适配某类任务行为、且线上可能需要维护多个场景适配器时，QLoRA 更合适。它把训练成本压低，同时保留了较强的适配能力。

## 5.7 延伸阅读

- Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models*: 理解低秩适配的原始思想。
- Dettmers et al., *QLoRA*: 理解低比特量化与 LoRA 结合后的工程价值。
- Hugging Face PEFT 官方文档：适合理解适配器如何在训练与部署中被管理。

## 5.8 小结

本章建立了模型改造的第一条主线：提示工程解决输入组织问题，SFT 解决任务行为问题，PEFT 在成本受限时提供了高性价比路径，LoRA 与 QLoRA 则是当前工程上最常见的适配方案。

## 5.9 下一章过渡

模型改造离不开数据。下一章将继续回答一个更基础但更决定上限的问题：训练数据从哪里来、如何组织、何时需要继续预训练，以及怎样让样本真正匹配模型训练目标。



# 第 6 章 数据工程与继续预训练：数据构建、样本拼接、增量预训练

## 内容提要

- ❑ 本章核心问题：为了继续改造企业知识助手，训练数据应该从哪里来、如何组织、何时用于 SFT、何时用于继续预训练？

❑ 主案例推进：为企业知识助手建立“知识文本线”“任务样本线”和“评测回流线”
- 三条数据资产主线。

❑ 读完后能做什么：看到一份原始企业材料时，知道它更适合进入检索库、SFT 样本池，还是继续预训练语料池，并知道进入前要经过哪些清洗与质检步骤。

## 6.1 学习目标

- 理解训练数据在 SFT、继续预训练和领域适配中的不同角色。
- 掌握样本清洗、格式化、样本拼接与增量预训练的基本策略。
- 能为企业知识助手建设一套可持续扩展、可追踪、可回滚的数据资产。

## 6.2 问题背景

模型工程的很多上限并不由训练框架决定，而由数据工程决定。没有结构清楚、边界明确的数据，微调只会把混乱放大。尤其是在企业场景中，文本来源复杂：制度文档、FAQ、工单记录、产品说明、会议纪要、客服对话都可能被拿来“喂模型”。如果不先建立数据分层，模型很快就会学到不一致风格、过期事实和模糊表达。

很多团队在这里犯的第一个错误，是把“有文本”误当成“有训练数据”。原始材料只是原料，不是天然可训练样本。第二个错误，是把所有问题都归到 SFT 或都归到继续预训练。更可靠的工程判断不是看名字，而是看症状：如果模型连领域文本都读不顺、术语分布不熟、日志和工单格式理解吃力，这更像语言分布问题；如果模型知识大体够用，但输出格式不稳、任务边界不清、口吻不符合企业要求，这更像任务行为问题。

还有一个常被忽略的事实是，所需数据量并不只由“模型多大”决定，更取决于原模型分布与目标分布的距离。如果企业材料和通用互联网文本风格接近，少量高质量样本就可能见效；如果材料充满内部缩略语、强结构字段、冷门术语和长尾流程，再多泛泛样本也不一定有用。

| 观察到的症状               | 更可能的问题类型  | 优先动作               |
|----------------------|-----------|--------------------|
| 回答内容基本对，但格式不稳、风格混乱   | 任务行为问题    | 先改提示模板，再补 SFT      |
| 制度名词、工单字段、日志格式读不顺    | 领域语言分布问题  | 优先考虑继续预训练          |
| 答案依赖最新制度、最新版本说明      | 动态知识问题    | 优先进入 RAG，而不是写死到参数里 |
| 多个子任务都不稳定，但都围绕同一输出规范 | 指令与样本设计问题 | 先收紧任务定义，再建设 SFT 样本 |

表 6.1: SFT、继续预训练与 RAG 的症状化判断

## 6.3 核心概念

### 6.3.1 三类数据资产

从用途上看，本书把数据分成三类：

1. **继续预训练数据**：用于让模型更熟悉领域语言分布。
2. **SFT 数据**：用于教模型完成具体任务、遵循输出格式。
3. **评测数据**：用于检验模型和系统是否真的变好。

这三类数据不能混为一谈。继续预训练强调规模和语料分布，SFT 强调指令与答案质量，评测数据强调代表性、稳定性和与训练链路的隔离。一本制度手册、一个 FAQ 页面、一次真实工单对话，都可能同时服务于这三类资产，但进入不同资产后的加工方式完全不同。

### 6.3.2 先分流，再构造：原始材料不是天然训练样本

同一份原始材料进入系统前，至少要先回答三个问题：它是用来被检索、教任务行为，还是让模型适应语言分布？如果这个问题没有先回答，后面的清洗、切分、训练和评测都会变得混乱。

对企业知识助手来说，更稳妥的方式是先做分流：

- **知识文本线**：原样保留事实与出处，优先服务 RAG。
- **任务样本线**：围绕目标任务构造输入输出对，服务 SFT。
- **语料适配线**：保留领域语言风格与结构分布，服务继续预训练。

这意味着“好数据”不是一个抽象标签，而是与用途绑定的好。适合进检索库的 PDF，不一定适合直接做 SFT；适合做继续预训练的海量规范文本，也不一定能直接作为高价值问答样本。

### 6.3.3 SFT 数据的三条硬标准

SFT 数据不是把企业文本改写成问答对就够了。它至少要满足三条硬标准：

1. **任务对齐**：样本必须直指目标任务，而不是“看起来像问答”。

2. **答案质量**：输出要体现团队希望模型学习的风格、边界和证据使用方式。

3. **格式稳定**：如果期望模型输出“结论-依据-建议动作”，训练样本就必须稳定体现这一规范。

很多低质量 SFT 数据的问题，不是答案完全错误，而是把模糊、含糊、风格混乱的表达也当作示范答案。模型会忠实学习这种不稳定，而不是自动替团队补齐规范。

### 6.3.4 领域语料来源优先级

继续预训练不要求每条语料都有标准答案，但非常依赖语料整体质量。旧稿里最有价值的一条经验是：优先选择知识密度高、语言质量稳、来源可控的文本，而不是为了规模先抓一切能抓到的内容。

| 语料来源                | 知识密度 | 可控性 | 建议                  |
|---------------------|------|-----|---------------------|
| 标准、制度、手册、教材、技术文档    | 高    | 高   | 继续预训练优先来源           |
| 清洗后的高质量网页、官方博客、产品文档 | 中高   | 中   | 可作为补充语料             |
| 论坛讨论、社交媒体、低门槛问答社区   | 波动大  | 低   | 谨慎使用，通常不作为主语料       |
| 原始聊天记录、工单对话、会议纪要    | 中    | 中   | 更适合做筛选后样本，不宜整批无差别入池 |

表 6.2: 继续预训练语料来源的优先级

### 6.3.5 数据不是越多越好，而是越可控越好

对工程项目而言，数据价值往往先取决于边界，再取决于规模。下面三件事比“先凑更多文本”更重要：

- 文本是否来自可信、可授权的来源。
- 样本是否真的对应目标任务，而不是把噪声一并带入。
- 数据是否能在版本上被追踪，出了问题能回滚。

继续预训练尤其如此。大量格式混乱、时间过期、事实互相矛盾的语料，不是“先练练看”，而是会先把模型污染，再增加后续定位问题的成本。

### 6.3.6 为什么要做样本拼接

在自回归训练中，很多原始文本样本都比较短。如果直接逐条训练，计算会浪费在 padding 和边界切换上。因此常见做法是把多个短样本按 token 长度拼接成更长序列，以提高训练效率。

但样本拼接并不等于“随便拼”。工程上最重要的一点，是意识到拼接带来的不仅是吞吐收益，也有伪上下文风险。如果把本不相关的问答、日志和制度片段硬拼到一起，模型会把它们误当成有意义的邻接关系，学到错误的语义共现。

### 6.3.7 样本拼接的三种常见策略

旧稿对样本拼接讲得比较散，这里把它压缩成三类最有工程价值的策略：

1. **随机拼接**：实现最简单，适合先解决 padding 浪费问题，但跨样本干扰最大。
2. **带分隔或分段 mask 的拼接**：通过边界标记或掩码，降低前后样本互相污染的风险。
3. **语义聚类拼接**：把相似文档或相似任务拼在一起，适合希望长上下文更“有意义”的场景，但也更容易带来重复和泄漏。

| 拼接策略         | 主要收益            | 主要风险           | 适用判断           |
|--------------|-----------------|----------------|----------------|
| 随机拼接         | 实现简单、节省 padding | 伪上下文最明显        | 先做基线时使用        |
| 边界标记或分段 mask | 降低跨样本干扰         | 预处理复杂度更高       | 希望兼顾吞吐和干净边界时使用 |
| 语义聚类拼接       | 局部长上下文更连贯       | 重复、泄漏、模板记忆风险更高 | 只有当相邻样本确实相关时使用 |

表 6.3: 样本拼接不是单一技巧，而是一组取舍

### 6.3.8 什么时候需要继续预训练

继续预训练适用于这样一类场景：基础模型对通用中文和英文都不错，但对企业领域文本的语言分布不够熟悉，例如内部术语、日志格式、工单字段、制度表达和产品名词大量出现。

如果问题主要是“风格不一致”或“任务格式不稳定”，优先考虑 SFT；如果问题主要是“模型连领域文本都读不顺”，继续预训练才更合适。还要注意，继续预训练不是知识更新的默认方案。凡是高频变化、版本敏感的事实，优先还是应该通过 RAG 接入。

### 6.3.9 继续预训练的数据量与混合策略

继续预训练最容易被问到的问题之一，是“到底需要多少数据”。更实用的回答不是报一个固定数字，而是看三件事：

- 目标语料与基座模型原始分布差距有多大。
- 企业术语和结构化表达的密度有多高。
- 目标是让模型“读顺文本”，还是同时承担下游行为迁移。

如果领域数据量不大，完全只喂垂域语料，反而可能导致灾难性遗忘。因此工程上常见做法是把领域语料与通用语料混合训练。旧稿里反复强调的一条经验是：可以把领域语料与通用语料按大约 1 : 5 到 1 : 10 作为起始尝试，但这只是经验起点，不是固定法则。真正重要的是持续看回归评测：领域能力有没有提升，通用能力有没有明显塌陷。

### 6.3.10 继续预训练中的训练信号

虽然本章不展开完整训练工程，但有几个与数据强相关的训练信号必须在这里先建立直觉。

第一，**loss 突刺**不一定意味着模型坏了。当语料分布突然变化、脏数据混入、学习率过高，或从旧检查点接着训练时没有重新 warmup，都会出现不正常的 loss 上冲。第二，**重新 warmup** 往往是继

续预训练的必要动作，因为此时模型面对的是一套新的语料分布，而不是完全延续原始预训练。第三，**batch size**、学习率和样本拼接方式是联动的，不能单看其中一个超参数。

## 6.4 主案例推进

在企业知识助手中，我们把数据建设分成三条线并行推进。

**第一条线是知识文本线：**收集制度、产品文档、操作手册、FAQ 和工单模板。这些数据一部分会进入 RAG 系统，一部分会作为继续预训练候选语料。

**第二条线是任务样本线：**围绕真实使用场景构造 SFT 数据，例如：

- 根据制度材料回答“是否允许”“如何申请”“谁负责”类问题。
- 根据工单上下文生成分类标签和下一步建议。
- 把长文档摘要成规定格式。

**第三条线是评测回流线：**把线上失败会话、人工抽检样本和关键制度问答整理成评测集，用于后续版本回归。

例如，一段制度原文可以先进入知识文本线，供后续 RAG 检索；再从中抽取一个高价值问答对，进入任务样本线，教模型学会“先给结论，再给依据”的回答形式；最后还可以从中挑出一批带版本号 and 标准答案的问题，进入评测回流线。也就是说，同一份原始材料可能服务于不同数据资产，但不能用同一种加工方式处理到底。

把这套流程写成工程步骤，会更清楚：

1. **来源审计：**确认版权、授权、时效性和版本号。
2. **文本清洗：**去广告、去模板噪声、统一编码、去重、过滤低质量片段。
3. **资产分流：**按用途进入 RAG、继续预训练或 SFT 流水线。
4. **版本登记：**记录数据来源、切分方式、处理脚本和模型版本。
5. **回归验证：**训练后用独立评测集看收益与副作用。

下面给出一个适合工程团队使用的数据质检表：

| 检查项     | 适用数据     | 通过标准              |
|---------|----------|-------------------|
| 来源可信与授权 | 全部数据     | 来源明确，可确认使用权限      |
| 时效性     | 知识文本、评测集 | 能标出版本或生效时间        |
| 任务对齐    | SFT 数据   | 样本直接对应目标任务，不是泛泛文本 |
| 结构完整    | 表格、制度、工单 | 关键信息没有在切分中丢失      |
| 版本可追踪   | 全部数据     | 能定位数据变更与模型版本对应关系  |
| 训练/评测隔离 | SFT、评测集  | 评测样本不回流进训练主集      |

如果要把“原始材料去向判断”再做得更细，可以参考下面这张分流表：



| 原始材料        | 默认去向          | 什么时候再转成 SFT | 补充说明           |
|-------------|---------------|-------------|----------------|
| 最新制度 PDF    | RAG / 评测线     | 需要固定回答格式时   | 动态知识优先保留为可检索文本 |
| 产品手册与操作手册   | RAG / 继续预训练   | 有高频标准问答时    | 语言分布稳定，适合进语料池  |
| 工单与客服对话     | SFT 候选 / 评测回流 | 经过清洗和脱敏后    | 不宜原样整批进继续预训练   |
| 内部术语词典、字段说明 | 继续预训练 / RAG   | 需要统一解释风格时   | 适合缓解术语读不顺问题    |

表 6.5: 企业原始材料到不同数据资产的分流

再看一个样本拼接示例。若有三条很短的制度问答样本，直接逐条训练会造成大量 padding；更合理的做法是按 token 长度把它们拼成一个较长序列，并在段落之间插入清晰边界标记。这样既能提高训练效率，也能降低模型误把前一个问答的风格延续到下一个样本中的风险。

如果团队已经确定要做继续预训练，还可以采用一种更贴近真实工程的两阶段路线：先用高质量领域文本做继续预训练，让模型适应企业语言分布；再用任务样本做 SFT，让模型学会回答边界与输出格式。某些场景下，也可以在继续预训练后段少量混入任务指令样本，作为连接二者的缓冲层，但前提是不能让任务样本反客为主，破坏继续预训练的语料目标。

## 6.5 常见坑与工程取舍

- 把所有企业文本都扔进继续预训练：如果语料噪声高、格式乱，会先污染模型再谈提升。
- 训练集和评测集来源不分离：得到的“提升”很可能只是记忆而非泛化。
- SFT 数据只追求数量：低质量自动生成数据会把模型推向模板化和幻觉化。
- 忽略版本管理：当回答质量突然变化时，如果无法定位是模型版本还是数据版本导致，就难以迭代。

再补四个旧稿里反复出现、但很容易被压掉的提醒：

- 只喂垂域语料会放大遗忘风险：领域能力可能上来，但通用能力和稳健性会塌。
- 语义聚类拼接不是越像越好：相似文本堆得太密，容易让模型记住模板而不是学会泛化。
- 不要把扩词表当领域适配捷径：除非确实证明分词切分极差，否则词表扩展未必值得。
- 继续预训练不看训练信号：loss 突刺、学习率过大、重启后不 rewarmup，都会把好语料练坏。

## 6.6 本章练习

 **练习 6.1 样本去向判断** 一份最新发布的差旅制度 PDF，应该优先进入哪条数据线？为什么不应默认先做 SFT 样本？

**解** 应优先进入知识文本线，因为它首先承担“被检索、被引用”的职责。只有当团队明确希望模型学习某种回答格式或高频问答行为时，才从中抽取高价值片段构造 SFT 样本。否则直接做 SFT 会把动态知识过早写进参数。

 **练习 6.2 继续预训练触发条件** 什么症状更能说明企业知识助手需要继续预训练，而不是只做 SFT？

**解** 如果模型连领域文本本身都读不顺，例如内部术语、日志格式、工单字段理解明显吃力，这更像领域语言分布不熟，需要继续预训练。若只是回答格式不稳或任务边界不清，则优先做 SFT。

## 6.7 延伸阅读

- **DeepLearning.AI / Hugging Face 的 LLM 数据课程资料**：适合理解数据清洗、切分与格式化的基本套路。
- **Together、MosaicML、Databricks 的继续预训练实践文档**：适合理解领域适配数据怎样进入训练流水线。
- **Cleanlab、Argilla 等数据质检工具资料**：适合理解数据错误如何被系统化发现。
- **Self-Instruct 与主动学习相关文章**：适合理解任务样本如何从原始材料中被逐步构造出来。

## 6.8 小结

本章回答了“模型改造靠什么喂出来”的问题。继续预训练解决领域语言分布问题，SFT 解决任务行为问题，而高质量的数据流水线是二者的共同前提。真正成熟的数据工程，不是把文本越堆越多，而是让每一份原始材料都知道自己该去哪里、以什么形态进入系统。

## 6.9 下一章过渡

当模型、提示词和任务数据逐渐成形后，企业知识助手仍然面临一个关键难题：很多答案并不适合写入参数，而必须实时从文档中获取。下一章将正式进入 RAG 基础，构建检索增强问答系统的最小闭环。



---

## 第三部分

# 构建知识系统

## 第 7 章 RAG 基础：检索增强问答系统的最小闭环

### 内容提要

- ❑ **本章核心问题：**当答案不能只依赖模型记忆时，怎样为企业知识助手搭建一个最小但真正可用的 RAG 闭环？
- ❑ **主案例推进：**让企业知识助手第一次真正“读企业文档”，从通用聊天工具迈向有依据的问答系统。
- ❑ **读完后能做什么：**能把 RAG 拆成离线建库与在线问答两段，知道错误首先出在哪一段、日志应该记什么、系统何时该拒答。

### 7.1 学习目标

- 理解为什么企业问答系统通常需要 RAG，而不是只靠模型记忆。
- 掌握最小 RAG 闭环：切分、索引、检索、拼接、生成。
- 能把企业知识助手从“聊天模型”升级为“有依据的问答系统”。

### 7.2 问题背景

到目前为止，我们已经可以调用模型、设计 prompt，也知道何时该微调。但企业场景的核心事实往往具有三个特征：**更新频繁、权限敏感、粒度细碎**。这决定了仅靠模型参数记忆并不可靠。制度今天更新、流程下周变更、产品说明分散在几十份文档中，这些知识更适合通过检索接入，而不是反复重训模型。

更重要的是，企业知识助手不仅要“像是知道”，还要**说得出依据、指出来源、允许复核**。这正是 RAG 的价值所在。它把“回答问题”拆成两件事：先找到可能的证据，再让模型基于证据组织语言。只要这两件事被分开，系统就第一次具备了可追溯和可复盘的基础。

### 7.3 核心概念

#### 7.3.1 什么是 RAG

RAG (Retrieval-Augmented Generation) 是“先检索、后生成”的系统范式。它不是一个单独算法，而是一条系统链路：

1. 把文档切分为可检索片段。
2. 为片段建立索引。
3. 根据问题召回相关内容。
4. 把召回结果拼接到 prompt 中。
5. 由生成模型基于这些材料作答。

从系统职责上看，RAG 不是“模型多了一个插件”，而是把原本都压在参数里的事情拆成了**知识接入**和**语言生成**两层。

### 7.3.2 RAG 的价值不只是补知识

工程上，RAG 至少带来四个价值：

- 降低知识时效问题。
- 提供引用依据与可追溯性。
- 缓解模型幻觉。
- 降低把动态知识写入参数的需求。

对企业知识助手来说，还应再补一层理解：RAG 让团队可以把“知识治理”独立出来。文档更新、版本回滚、权限切分、来源登记，这些原本很难通过训练参数管理的能力，都能通过知识库层面显式治理。

### 7.3.3 什么时候优先上 RAG

一个很实用的判断标准是：如果问题的正确答案取决于最近更新的文档、部门权限、流程细节或多份企业材料的组合，就应该优先考虑 RAG，而不是继续逼模型“记住更多”。这类问题本质上不是语言生成问题，而是知识接入问题。

反过来说，如果问题主要是“回答格式不稳”“口吻不符合企业规范”，那就更像 SFT 或提示工程的职责。RAG 不是所有问题的答案，它只在“需要实时接入知识”这一类问题上最有价值。

### 7.3.4 最小闭环中的离线段与在线段

把 RAG 真正做成系统时，最好先把它拆成两段。

**离线段**负责知识准备，包括：文档收集、清洗、切分、向量化、索引构建和元数据登记。它决定系统后面有没有可能拿到正确证据。

**在线段**负责回答生成，包括：接收问题、检索 top-k、构造 prompt、生成回答、输出引用和记录日志。它决定系统最终如何把证据组织成用户能用的答案。

这个切分非常重要，因为很多团队会把全部问题都归到“模型不够强”。实际上，离线段一旦做错，在线段再怎么调也只能在坏证据上作答。

### 7.3.5 文档切分不是技术细节，而是语义边界设计

最小闭环里最容易被低估的一步是切分。一个片段切得过长，会把无关内容一并带进检索；切得过短，又会把表头、定义、限制条件和例外条款拆散。工程上至少要在三个方向上做判断：

- **固定长度切分**：实现简单，但容易破坏结构。
- **按标题和段落切分**：更保留文档语义，但依赖原文结构质量。
- **按结构块切分**：对表格、流程和 FAQ 更友好，但预处理更复杂。

最小闭环阶段不一定要追求最复杂的切分器，但一定要保住两个东西：片段语义尽量完整，以及片段和原文位置的对应关系清楚可追踪。

### 7.3.6 检索结果为什么必须带元数据

RAG 里的片段不应该只是一段裸文本。至少要带上来源文档、章节、页码或段落编号等元数据。这样做有三层好处：

- 方便模型输出引用。
- 方便人工排查错误。
- 方便知识库更新后做回滚和重建。

没有元数据的 RAG 系统，表面上也能“答题”，但一旦答案有争议，团队几乎无法复盘“模型到底看到了什么”。

7.3.7 提示拼接与保守回答

检索出来的片段并不会自动变成好答案。系统还必须明确告诉模型：哪些是材料，哪些是问题，回答是否必须引用，证据不足时应怎样处理。

一个真正可上线的企业知识助手，不能只会“根据材料回答”，还要会“根据材料拒答”。也就是说，当检索片段不足以支撑结论时，系统应允许模型回答“依据不足，建议补充某类材料”或“当前材料无法判断”。这种保守机制不是弱，而是企业场景的必要能力。

7.3.8 如何区分检索失败与生成失败

RAG 最实用的工程直觉之一，是把错误先归到哪一层。一个简单的判断方式是：

- 如果召回片段本身就不相关，优先判为**检索失败**。
- 如果召回片段相关，但最终回答没有忠实使用证据，优先判为**生成失败**。
- 如果片段相关但被截断、顺序混乱或没进 prompt，优先判为**系统拼接失败**。

只要团队能稳定做出这一步判断，后续优化就会清晰很多。

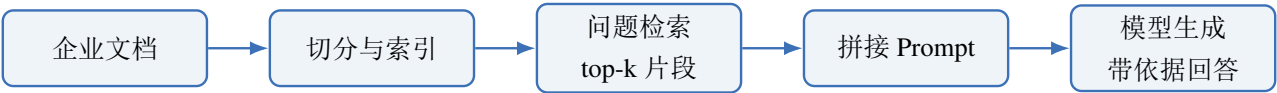


图 7.1: 企业知识助手的最小 RAG 闭环

| 环节        | 最小设计重点          | 最先暴露的问题            |
|-----------|-----------------|--------------------|
| 切分与索引     | 保住语义边界和来源元数据    | 片段不完整、表头丢失、页码丢失    |
| 检索        | 让相关片段稳定进 top-k  | 召回不到关键条款、召回噪声太多    |
| Prompt 拼接 | 明确材料、问题、引用和拒答规则 | 片段顺序乱、材料没真正进入模型上下文 |
| 生成        | 忠于材料组织答案        | 胡乱总结、无依据扩写、拒答不保守   |

表 7.1: 最小 RAG 闭环中每一段真正要守住的东西

```
retrieved_docs = retriever.invoke(question)
context = "\n\n".join(doc.page_content for doc in retrieved_docs[:4])

prompt = f"""
你是企业知识助手。请仅依据给定材料回答。
```

若材料不足，请明确说明“依据不足”。

材料：

```
{context}
```

问题：

```
{question}
```

```
"""
```

这个最小示例看起来很简单，但它已经把 RAG 最重要的接口暴露出来了：问题是什么、召回了什么、最终把什么材料送进模型。只要这三者能被记录和回放，系统就具备了继续优化的基础。

## 7.4 主案例推进

在主案例中，RAG 的第一阶段目标不是做复杂优化，而是做到三件事：

- 能从制度文档和 FAQ 中检索到与问题最相关的片段。
- 回答时明确引用这些片段，而不是只输出结论。
- 在材料不足时明确说“不足以判断”，建立保守回答机制。

这一阶段的系统架构应当尽量简单：离线建索引，在线检索 top-k，拼接到固定提示模板中，再调用基础模型生成答案。只有当这一最小闭环可用后，后续的 PDF 解析、重排序、GraphRAG 等优化才值得引入。

如果把企业知识助手的最小上线过程写成更工程化的清单，可以是：

1. 先选一批高价值文档：制度、FAQ、产品手册。
2. 设计统一切分与元数据规则，让每个片段都可回溯。
3. 用最简单的检索器先跑通 top-k 召回，不急着堆复杂技巧。
4. 设计固定回答模板，明确依据不足时的拒答方式。
5. 保存问题、检索片段、最终 prompt 和回答，形成回放日志。

对刚起步的团队来说，日志比 fancy 架构更重要。因为只有当“问题-片段-prompt-回答”这条链完整留下来，团队才能在下一轮知道自己该改切分、改检索还是改生成。

## 7.5 常见坑与工程取舍

- 一开始就追求复杂 RAG 架构：最小闭环没跑通前，复杂优化通常只会放大不确定性。
- 检索结果不进日志：如果看不到召回片段，就无法判断问题出在检索还是生成。
- top-k 固定过大：上下文变长未必更准确，很多时候只是把答案变得更模糊。
- 不处理“无依据”场景：系统会为了“看起来有用”而编造答案。

再补四个最小闭环阶段特别容易踩的坑：

- 只测演示样例，不测真实问法：上线后最先出问题的往往是口语化提问。
- 把所有文档一股脑入库：低价值、过期、重复文档会迅速拉高检索噪声。
- 不区分离线建库与在线问答责任：坏片段和坏回答最终混成一团，无法归因。
- 把 RAG 当作“自动更准确”：检索只是给模型材料，不等于模型一定会忠实使用。

## 7.6 本章练习

🔥 **练习 7.1 检索失败还是生成失败** 企业知识助手回答错误时，日志显示召回的四个片段都与问题无关。此时问题首先属于哪一层？下一步应该优先检查什么？

**解** 这首先属于检索层问题。下一步应优先检查文档切分、索引质量、查询表达和 top-k 设置，而不是先去调生成模型的温度或做 SFT。

🔥 **练习 7.2 为什么先做最小闭环** 为什么在企业知识助手里，不应在最小 RAG 闭环还没跑通时就急着上重排序、GraphRAG 或复杂 agent 流程？

**解** 因为最小闭环没跑通时，系统连“证据是否被召回、是否真的进入 prompt、材料不足时是否保守回答”这三件基础事实都无法保证。此时再叠复杂模块，只会把错误来源变得更难定位。

## 7.7 延伸阅读

- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*: RAG 名称的经典来源。
- LlamaIndex / LangChain 的 RAG 入门文档：适理解最小闭环如何在框架里实现。
- Pinecone、Weaviate、Milvus 的检索实践文档：适理解向量检索在系统里承担的角色。
- 企业知识库治理与文档生命周期资料：适理解为什么 RAG 的上限不仅由模型决定，也由知识源治理决定。

## 7.8 小结

RAG 基础的关键不是某个具体框架，而是建立“可检索、可引用、可复盘”的知识接入闭环。企业知识助手一旦具备这一闭环，就从通用聊天工具迈向了真正的知识系统；而这个闭环是否稳，首先取决于切分、元数据、日志和拒答这几件看似朴素的基础工作。

## 7.9 下一章过渡

最小闭环建立后，新的瓶颈会很快出现：PDF 解析不稳、表格信息难提取、召回片段不够准、查询表达太口语化。下一章将集中处理这些文档理解与检索优化问题。

## 第 8 章 文档理解与检索优化：PDF、表格、召回增强与 GraphRAG

### 内容提要

- ❑ **本章核心问题：**当最小 RAG 已经跑通但效果仍不稳定时，应该优先优化文档理解、召回增强还是知识结构，判断顺序是什么？
- ❑ **主案例推进：**把企业知识助手从“能检索”推进到“能稳定检索”，解决真实企业文档脏、乱、杂的问题。
- ❑ **读完后能做什么：**看到召回片段不准或文本混乱时，能先判断问题在 PDF 解析、切分、查询增强，还是图结构。

### 8.1 学习目标

- 理解 RAG 系统中常见的输入侧与检索侧瓶颈。
- 掌握文档解析、表格识别、查询增强、负样本挖掘、RAG-Fusion 与 GraphRAG 的使用边界。
- 能为企业知识助手设计一套分层优化路线，而不是堆砌技巧。

### 8.2 问题背景

最小 RAG 闭环通常只能解决“文档比较整洁、问题表达比较标准”的场景。一旦进入真实企业环境，系统会遇到更复杂的输入：扫描件 PDF、双栏版面、页眉页脚、复杂表格、口语化问题以及大量“看起来相关但实际上没用”的召回片段。

这时团队很容易误以为“检索不准就是嵌入模型不够强”。事实上，RAG 优化最常见的误判，恰恰是把输入理解问题、召回问题和知识结构问题混成一团。第 8 章的目标，就是把这些问题重新分层，让优化顺序可判断、可回退、可停手。

### 8.3 核心概念

#### 8.3.1 文档理解先于检索优化

如果原始文档被错误解析，再好的嵌入模型也无能为力。因此优化顺序通常应该是：

1. 先解决文档解析质量。
2. 再解决切分与结构保留。
3. 最后再做召回增强和重排序。

这条顺序看起来朴素，却能避免大量无效迭代。因为一旦解析阶段已经把表头、标题层级、页码和阅读顺序打乱，后面所有“检索优化”都只是在坏输入上调参。



### 8.3.2 为什么 PDF 与表格单独成问题

PDF 更像“排版结果”，而不是天然的语义结构。版面分析、标题层级、阅读顺序和表格边界都可能丢失。表格则额外引入了行列关系：如果切分只按文本长度进行，表头与单元格内容很容易被拆散，导致检索片段失去意义。

对企业知识助手来说，制度文件、报价说明、操作手册和审批流程表大量存在，因此“能不能把 PDF 变成可检索知识”本身就是一条独立能力线，而不是附属预处理。

### 8.3.3 结构保留比“切得均匀”更重要

文档切分的目标不是让片段看起来长度差不多，而是让**语义单位尽可能完整**。在真实企业文档里，下面几类结构特别值得保留：

- 标题与小节的层级关系。
- 表格中的表头、列名与单元格邻接关系。
- FAQ 中的问题和对应答案边界。
- 制度中的主规则、例外条件和适用范围。

如果这些边界在切分时丢失，检索系统虽然还能返回“相关关键词”，但对用户真正有用的上下文往往已经被拆坏了。

### 8.3.4 查询增强：让问题更像文档语言

当用户问题过于口语化时，可以先把问题改写成更适合检索的表达。典型做法包括：

- **HyDE**：先让模型生成一段假想答案，再用这段答案去检索。
- **多查询扩展**：从一个问题生成若干检索子问题。
- **RAG-Fusion**：融合多个查询的召回结果，再排序合并。

这些方法本质上都在解决同一个矛盾：**用户语言**和**文档语言**未必是一回事。员工会说“报销住宿有啥限制”，文档里写的却可能是“差旅标准”“住宿补贴上限”“例外审批条件”。查询增强做的，就是弥合这种表达差距。

### 8.3.5 负样本与重排序

当系统规模变大后，通用嵌入模型可能不够贴合领域分布。这时可以通过构造难负样本来训练或微调检索器，让它更好地区分“看起来相关”与“真正有用”的片段。

与此同时，重排序器的职责也要讲清楚：检索器更像**先把候选范围缩小**，重排序器更像在**候选中重新排优先级**。当系统经常“召回到了相关材料，但不是最关键那一段”时，重排序比一味扩 top-k 更常是更好的动作。

### 8.3.6 GraphRAG 的边界

GraphRAG 适合处理实体、关系和多跳结构明显的知识，例如组织架构、设备依赖关系、系统组件关系等。它不适合被当成所有 RAG 问题的统一答案。很多企业知识问答的主要瓶颈仍然是文档解析与召回质量，而不是缺少图结构。

工程上可以用一个简单判断来避免误用：如果用户问题主要依赖“谁与谁有什么关系、哪一步依赖哪一步”，图结构可能有价值；如果问题主要依赖“文档里到底怎么写”，那大多数时候应该先把文档理解和召回做好。

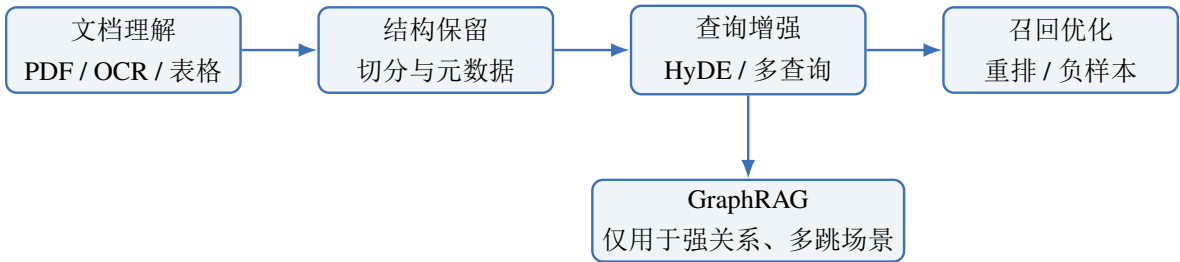


图 8.1: 文档理解与召回增强的优化顺序

8.3.7 从症状判断该优先修哪里

为了避免把优化方法堆成清单，可以用下面的判断顺序：

- 召回片段文字本身就乱：先查 PDF 解析、OCR 和切分。
- 召回到的是“相关但不”的片段：先查查询表达、重排序和难负样本。
- 跨表格、跨字段信息经常丢失：先查结构保留方式。
- 问题天然依赖组织关系或多跳依赖：再考虑 GraphRAG。

| 观察到的现象             | 优先怀疑哪一层 | 第一动作               |
|--------------------|---------|--------------------|
| 片段出现页眉页脚、断裂句子、表格残片 | 文档理解层   | 先修解析和切分，不急着换嵌入模型   |
| 召回“像相关”但答案支撑力弱     | 召回层     | 先上重排序或查难负样本        |
| 用户口语化提问命中率         | 查询层     | 先试问题改写、HyDE 或多查询   |
| 组织链路、多跳依赖问题常错      | 知识结构层   | 再考虑 GraphRAG 或关系抽取 |

表 8.1: 从症状倒推 RAG 优化入口

8.4 主案例推进

在主案例中，我们按以下顺序推进优化：

1. 对制度 PDF、操作手册和项目文档建立稳定的解析规则。
  2. 对表格类文档保留表头、列名和行上下文，避免语义断裂。
  3. 为口语化查询加入轻量问题改写或 HyDE。
  4. 当召回误伤明显时，再考虑重排序或难负样本训练。
  5. 只有在组织关系、流程依赖这类强关系问题中，再引入 GraphRAG。
- 这一顺序的核心思想是：先修输入质量，再修召回质量，最后才引入更重的知识结构。如果把每一步该产出的结果写得更工程化，可以得到一张“优化停手表”：

| 优化层       | 主要收益          | 何时继续往下走         | 何时先停手            |
|-----------|---------------|-----------------|------------------|
| 文档解析与结构保留 | 让片段可读、可用、可定位  | 片段已经基本干净，但召回仍不准 | 解析结果还一团乱时        |
| 查询增强      | 缩小用户语言与文档语言差距 | 口语化问题仍频繁漏召回     | 召回问题其实来自坏片段时     |
| 重排序/难负样本  | 提高候选片段排序质量    | 召回到了相关材料但顺序总不对  | top-k 本身就没召回到证据时 |
| GraphRAG  | 处理实体关系和多跳结构   | 问题明显依赖关系图       | 文档问答本质还是单跳查证时    |

表 8.2: RAG 优化需要退出条件，而不是无限堆技巧

企业知识助手在这个阶段最需要的，不是“把所有先进方法都试一遍”，而是每引入一种方法，都知道它解决的到底是哪一层问题。如果连这一点都说不清，优化很容易变成技术名词堆砌。

### 8.5 常见坑与工程取舍

- **解析错误却去调嵌入模型**：这是 RAG 项目中最常见的误诊之一。
- **为所有问题都启用多查询扩展**：召回率可能提升，但延迟和成本也会显著增加。
- **把 GraphRAG 当潮流组件**：如果知识本身没有明显实体关系图，引入图结构只会增加复杂度。
- **忽略优化顺序**：同时改解析、切分、召回和生成，最后很难知道哪一步真的起作用。

再补四个在企业文档里特别典型的误区：

- **只保留正文，不保留元数据**：页码、章节、表头一丢，引用和回溯都会变差。
- **把表格当普通段落切分**：数值关系和字段约束会被切碎。
- **召回率上来就认为系统变好**：噪声片段也可能一起上来，最终答案反而更乱。
- **没有负样本意识**：检索器就会把“有相同词”误当成“真相关”。

### 8.6 本章练习

 **练习 8.1 文档解析失败** 某制度 PDF 的召回片段中经常混入页眉页脚和断裂表格，导致答案看起来“引用了材料”但实际不可读。此时最优先的优化方向是什么？

**解** 最优先的方向是**文档理解与结构保留**，也就是修 PDF 解析、OCR 和表格结构提取，而不是先换嵌入模型或上 GraphRAG。因为当前问题是输入质量在进入检索前就已经损坏。

 **练习 8.2 何时考虑 GraphRAG** 什么样的问题更适合考虑 GraphRAG，而不是继续堆查询增强？

**解** 当问题天然依赖**实体关系和多跳推理**，例如组织架构、审批链路、设备依赖关系时，更适合考虑 GraphRAG。若只是用户表达口语化或文档解析不稳，继续堆查询增强通常不是最优先方案。

### 8.7 延伸阅读

- **HyDE、RAG-Fusion 相关论文与开源实现**：适合理解查询增强为什么能弥合“用户语言”和“文档语言”之间的差距。

- **Unstructured、MinerU、Nougat、Docling** 等文档解析工具文档：适合理解 PDF 与表格处理的工程现实。
- **GraphRAG 官方资料与案例**：适合理解图结构真正适合什么问题，而不是把它当通用组件。
- **检索器微调与难负样本实践资料**：适合理解“相关但无用”的召回为什么需要专门治理。

## 8.8 小结

RAG 优化不是一串彼此独立的小技巧，而是一条输入理解与召回增强的分层路线。真正有工程价值的优化，必须能回答三个问题：它修的是哪一层、为什么现在修这层、什么情况下可以先停手。

## 8.9 下一章过渡

当系统逐渐变复杂，团队必须回答另一个问题：到底是哪个环节在变好，哪个环节在拖后腿？下一章将建立评测体系，把模型效果、检索质量、系统性能和人工反馈统一纳入度量。

## 第 9 章 评测体系：模型、系统与应用层评测

### 内容提要

- ❑ **本章核心问题：**如何判断一次系统改动到底让企业知识助手变好了，还是只是把失败方式换了一种表现？
- ❑ **主案例推进：**为企业知识助手建立从检索、生成到成本和人工反馈的评测闭环。
- ❑ **读完后能做什么：**看到版本分数变化时，能先定位问题属于模型层、检索层、系统层还是业务层，而不是只盯总分。

### 9.1 学习目标

- 理解为什么 LLM 项目必须建立分层评测，而不能只看主观体验。
- 掌握模型层、检索层、系统层和业务层的主要指标与切片方法。
- 为企业知识助手搭建可持续复用、可解释、可回归的评测闭环。

### 9.2 问题背景

在 LLM 项目中，“这次回答看起来不错”并不是可靠结论。模型输出带有较强随机性，不同配置之间还存在风格差异、长度差异和任务分布差异。如果没有评测体系，团队很容易陷入以下循环：改了 prompt，感觉好了一点；换了嵌入模型，又感觉差了一点；加了重排序，似乎又变好了。这样的项目无法稳定迭代。

企业知识助手比通用聊天更需要评测，是因为它不仅需要“答得像回事”，还要满足引用规则、权限边界、时效性要求和成本约束。一次改动可能提升了回答流畅度，却让引用更不准确；也可能让召回更全，却把延迟和成本推到了不可上线的程度。因此，评测体系真正要回答的不是“总分有没有上升”，而是哪一层变好了，哪一层变坏了，代价是什么，风险在哪里。

### 9.3 核心概念

#### 9.3.1 为什么要做分层评测

同一个失败样本可能来自不同层：

- **模型层：**模型推理能力不够或格式遵循差。
- **检索层：**该召回的材料没有召回，或召回了不该召回的噪声。
- **系统层：**上下文拼接错误、超长截断、服务超时或缓存失效。
- **业务层：**回答虽语义正确，但不符合企业流程、合规或权限要求。

因此评测必须分层，否则“整体分数下降”无法指导任何修复动作。对 RAG 系统来说，还要额外检查四项关键能力：

- **抗噪声能力：**混入无关片段后，系统是否还能抓住真证据。
- **拒答能力：**证据不足时，能否保守地说“不知道”或“需要更多信息”。

- **跨文档综合能力**：答案是否真正整合了多份材料，而不是只复述单一片段。
- **反事实稳健性**：当检索上下文里混入误导信息时，系统会不会被带偏。

9.3.2 评测集不是一个集合，而是一组切片

很多团队把“评测集”想成一个固定的 Excel 表，这太薄了。更合理的做法是把评测集分成多个切片，每个切片负责暴露一种风险。常见切片包括：

- **基础问答切片**：覆盖高频、低歧义、业务价值高的问题。
- **困难样本切片**：覆盖长问题、口语化提问、长尾术语和跨文档问题。
- **拒答切片**：故意提供证据不足、权限不足或问题表达不全的样本。
- **线上回流切片**：从真实失败会话中抽样，防止团队只对“旧题库”优化。

切片化的价值在于，版本分数上升时，团队能知道“上升发生在哪些子场景”；版本分数下降时，也能知道“是哪个角落先坏了”。

9.3.3 RAG 评测要拆成独立评测与端到端评测

旧稿里一个非常值得保留的区分，是把 RAG 评测拆成**独立评测**和**端到端评测**。

**独立评测**关注检索与证据链本身是否正常，例如召回的文档对不对、重排是否把关键片段放到了前面、引用是否覆盖了答案所需证据。它的好处是能快速定位“证据没拿到”这一类问题。

**端到端评测**则把检索、上下文拼接、生成和回答策略一起评估，直接看最终答案是否正确、是否忠实于证据、是否满足业务要求。它更接近用户体验，但定位原因更难。

| 评测方式  | 主要回答什么问题              | 更适合什么场景            |
|-------|-----------------------|--------------------|
| 独立评测  | 证据有没有召回、重排有没有把关键片段放前面 | 调嵌入、切分、召回、重排时优先使用  |
| 端到端评测 | 用户最终拿到的答案是否正确、忠实、可用   | 做版本发布、业务验收和整体对比时必做 |

表 9.1: RAG 独立评测与端到端评测的分工

9.3.4 检索指标与系统指标

对于检索层，常见指标包括 Precision@k、Recall@k 和 MRR。对企业知识助手来说，Recall@k 往往比单纯的生成分数更能解释问题，因为很多失败根本不是“不会答”，而是“没拿到证据”。

如果是 RAG 系统，还可以把几个常见指标理解成在回答不同问题：



| 指标                 | 主要回答的问题           | 典型含义         |
|--------------------|-------------------|--------------|
| Answer Relevancy   | 答案有没有对准用户问题       | 避免答得流畅但答偏了   |
| Faithfulness       | 答案是否忠于检索到的证据      | 防止模型“有证据也乱编” |
| Context Precision  | 送进模型的上下文里有多少是真相关的 | 衡量检索噪声是否过大   |
| Answer Correctness | 最终答案与基准答案是否一致     | 更偏最终任务结果     |

表 9.2: RAG 常见指标各自回答的问题

系统层则必须同时看效果与成本，至少持续记录：

- 平均延迟和 P95 延迟。
- 输入输出 token 数。
- 单次调用成本。
- 失败率、超时率和重试率。

### 9.3.5 先定位，再打分

评测最容易失效的地方在于“只给总分，不解释问题在哪”。更实用的顺序是：

1. 先看失败样本属于模型层、检索层、系统层还是业务层。
2. 再为对应层选择指标，而不是先找一个统一分数。
3. 最后才把分层结果汇总成版本对比。

把这件事做成流程，会比口号更有用：

1. **建集**：先为目标任务建立基础集、困难集、拒答集和线上回流集。
2. **设基线**：记录当前版本在各切片上的结果，而不是只有一个总分。
3. **跑自动评测**：先做大规模筛查，看哪些层面出了变化。
4. **抽样人工评测**：确认自动指标没看错方向，尤其检查保守性、引用合规与业务可用性。
5. **失败分析**：把失败样本按层级和场景归因。
6. **回填回归集**：把线上新增失败模式加入后续评测切片。

### 9.3.6 人工评测依然重要

即使有自动指标，人工评测仍不可替代，因为很多企业任务关心的是真实可用性，例如：

- 回答是否足够保守。
- 是否遵守引用规则。
- 是否会给出越权建议。
- 是否真正减少了员工搜索文档的时间。

可以把自动评测和人工评测的分工理解成“一个负责规模，一个负责语境”：



| 评测方式 | 更擅长做什么             | 主要局限               |
|------|--------------------|--------------------|
| 自动评测 | 大规模回归、快速对比、持续监控    | 难以完整理解业务风险和真实可用性   |
| 人工评测 | 判断保守性、引用合规、企业流程匹配度 | 成本高、规模受限、标准一致性需要设计 |

表 9.3: 自动评测与人工评测的分工边界

当两个版本分数很接近、单项指标也各有胜负时，直接打绝对分往往很痛苦。这时可以借鉴“两两对比”的思路，让评审者只回答“版本 A 和版本 B 谁更好”，再汇总成相对排序。它不一定要做成正式 ELO 系统，但这个思路很适合解决“两个版本都不完美，但必须选一个上线”的现实问题。

## 9.4 主案例推进

在主案例中，我们把评测分成四套集合：

1. **基础问答集**：覆盖制度、FAQ、产品说明等高频问题。
2. **困难样本集**：覆盖长问题、口语化提问、跨文档问题和依据不足问题。
3. **检索诊断集**：重点检查召回片段是否正确、是否缺关键证据。
4. **线上回流集**：从真实失败会话中定期抽样纳入回归测试。

测试集建设的原则也要显式写出来，否则后面很容易“建成题库，建不成评测体系”：

- **多样性**：不要只收集标准问句，还要覆盖口语化、模糊化和跨文档提问。
- **真实性**：优先收集真实业务问题，而不是只靠想象构造。
- **难度分层**：让团队知道版本是在简单题上进步，还是在难题上进步。
- **训练/评测隔离**：评测样本不回流进训练主集。
- **失败回流**：线上问题必须周期性进入回归集。

下面这张评测矩阵表能帮助团队避免“只有总分”的误区：

| 评测层 | 关注问题          | 代表指标                | 典型修复动作               |
|-----|---------------|---------------------|----------------------|
| 模型层 | 会不会答、格式稳不稳    | 任务正确率、格式通过率         | 改 prompt、做 SFT、改模型选择 |
| 检索层 | 证据能不能召回       | Recall@k、MRR、重排命中率  | 改切分、嵌入、查询增强、重排       |
| 系统层 | 成本与稳定性是否可接受   | P95 延迟、失败率、token 成本 | 控上下文、量化、缓存、服务调度      |
| 业务层 | 是否符合真实流程与风险要求 | 人工评分、业务通过率          | 改规则、权限、审批链与回答策略      |

每次改动都应回答四个问题：正确率是否提升、依据质量是否提升、延迟和成本是否恶化、是否在某个子场景上出现退化。

如果团队希望把评测做成真正的平台化流程，那么自动化测试、人工评测、线上监控、安全检查和性能基准就不应是五套分散的脚本，而应是一条可重复的流水线。理想状态下，每次改 prompt、换

检索器、改缓存策略、上量化或加重排，都能触发一套统一的版本对比流程。

## 9.5 常见坑与工程取舍

- 只做单一总分：总分高不代表企业最看重的场景真的变好。
  - 评测集长期不更新：系统会逐渐“学会”旧评测，而忽视新问题。
  - 没有失败样本回流机制：线上踩过的坑会一遍遍重复出现。
  - 只看效果，不看成本和延迟：很多“最优方案”实际上无法上线。
- 再补四个很常见的评测失真来源：
- 训练与评测污染：最近刚拿来做人 SFT 的样本，又出现在评测集里。
  - 评估标准不一致：不同人工评审对“可接受答案”的标准完全不同。
  - 只做端到端，不做独立诊断：系统坏了，但团队不知道是检索坏了还是生成坏了。
  - 盲信自动裁判：LLM-as-a-Judge 很有用，但它不是业务真理本身。

## 9.6 本章练习

 **练习 9.1 指标设计** 团队把新版本的“整体主观满意度”从 7.8 提升到 8.0，就宣布 RAG 优化成功。为什么这个结论不够稳？

**解** 因为这只给出了单一总分，没有说明提升来自模型层、检索层还是系统层，也没有说明延迟和成本是否恶化。RAG 优化至少还应检查检索诊断集的 Recall@k、引用质量和 P95 延迟，否则无法证明改动真的解决了知识接入问题。

 **练习 9.2 测试集污染** 为什么企业知识助手的评测集不应直接复用最近做人 SFT 的训练样本？

**解** 因为这样容易出现训练与评测不分离，系统看起来“变好”可能只是记住了样本，而不是对真实问题有更好泛化。评测集必须保持独立，才能反映真实迭代质量。

## 9.7 延伸阅读

- **RAGAS、TruLens、DeepEval 等评测工具资料**：适合理解 RAG 和应用层评测怎样落地。
- **OpenAI Evals 与各类 benchmark 框架文档**：适合理解“版本对比”如何被系统化执行。
- **Chatbot Arena 与两两对比类评测思路**：适合理解难分高下的版本如何做相对排序。
- **Google、Microsoft 等生产系统案例中的运维指标说明**：适合理解效果指标之外的稳定性要求。

## 9.8 小结

评测体系的作用不是证明模型优秀，而是让每次改动都可以被解释、被比较、被复盘。对工程团队而言，真正有价值的不是一个漂亮分数，而是知道问题在哪一层、该修什么、修完有没有副作用。

## 9.9 下一章过渡

当问答系统具备了可评测性，团队就可以更有把握地进入更强的模型行为控制手段。下一章将讨论对齐与强化学习，回答“如何让模型不仅会答，还能按我们希望的方式去答”。

---

## 第四部分

# 对齐、优化与规模化工程

## 第 10 章 对齐与强化学习：RLHF、PPO 与 NLP 中的 RL

### 内容提要

- ❑ 本章核心问题：除了让模型“答对”，还应该怎样让它“按希望的方式回答”，以及 RLHF 在这个过程中到底解决什么问题？
- ❑ 主案例推进：把企业知识助手从“会完成任务”推进到“更符合偏好与风险约束”。
- ❑ 读完后能做什么：知道什么时候 RLHF 值得做，什么时候它只是掩盖基础问题的错误方向。

### 10.1 学习目标

- 理解为什么仅靠 SFT 仍然不足以完全控制模型行为。
- 掌握 RLHF 的基本流程，以及 PPO 在其中的角色。
- 能从工程角度判断何时需要对齐训练，而不是把它神化。

### 10.2 问题背景

SFT 可以让模型学会“像训练答案那样输出”，但在真实交互中，团队往往还关心更复杂的行为：回答是否更安全、更简洁、更保守，是否会优先给依据，是否会在模糊问题上主动澄清。这些目标很难通过一组标准答案完整表达，因此出现了对齐训练的需求。

从工程角度看，对齐问题的本质不是“模型会不会做题”，而是“模型在多种可行答案里，是否更偏向我们接受的那一种”。企业知识助手尤其如此。很多时候，不同回答在事实层面都不算错，但在保守性、引用规范、风险提示、权限边界和行动建议上差异很大。只靠标准 SFT 样本，很难把这种偏好完整编码进去。

### 10.3 核心概念

#### 10.3.1 为什么需要对齐

SFT 的优化目标是最大化标准答案的似然，但真实偏好往往是相对性的。人类更容易判断“两段回答哪一段更好”，而不是为每个问题写出唯一正确答案。对齐训练就是把这种相对偏好引入模型优化过程。

可以把模型优化目标分成三层来看：

- 能力层：模型会不会做这件事。
- 行为层：模型做这件事时，是否按指定风格和边界输出。
- 策略层：在多种可行行为中，模型优先选择哪一种。

SFT 主要解决前两层，而 RLHF 更偏向解决第三层。也正因如此，它通常不应替代基础能力建设，而应建立在基础能力已经足够可用的前提上。

### 10.3.2 RL 的最小概念集

对工程读者来说，不需要把强化学习学成完整课程，但至少要建立四个最小概念：

- **策略**：模型当前的行为规则，也就是“给定输入时倾向输出什么”。
- **奖励**：系统对行为结果的偏好信号。
- **轨迹**：一次输入到输出的完整过程。
- **策略更新**：根据奖励调整模型，让高奖励行为更容易再次出现。

在大语言模型里，这些概念并不是拿来玩游戏，而是用来回答一个更实务的问题：**模型已经会答了，怎样让它更稳定地答成我们想要的样子。**

### 10.3.3 RLHF 的基本流程

典型 RLHF 流程包括三步：

1. 先做 SFT，得到能完成任务的基础策略。
2. 训练奖励模型，学习人类对不同回答的偏好。
3. 用强化学习方法优化策略，使模型倾向于产生高奖励回答。

这里最关键的转折点，在于训练信号从“标准答案是什么”变成了“这两个答案哪个更好”。这让模型开始学习偏好，而不仅仅是学习模板。

### 10.3.4 偏好数据与奖励模型

奖励模型并不是凭空出现的，它来自偏好数据。偏好数据最常见的形式，不是长篇标注说明，而是成对比较：给定同一个问题，让标注者选择答案 A 和答案 B 谁更好。

| 数据形式      | 更容易表达什么            | 主要风险          |
|-----------|--------------------|---------------|
| 标准 SFT 答案 | 任务格式、固定写法、基础行为     | 容易把偏好问题压成单一模板 |
| 成对偏好比较    | 哪个答案更保守、更有依据、更符合风格 | 标注标准不清时容易漂移   |
| 线上反馈回流    | 真实使用中的好坏判断         | 数据噪声大，需要清洗和分层 |

**表 10.1:** SFT 数据、偏好数据和线上反馈的职责不同

对企业知识助手来说，偏好数据特别适合表达下面这类差异：

- 同样都引用了材料，但哪个答案更保守。
- 同样都答对了，哪个答案更先给结论再给依据。
- 同样都不完整，哪个答案更愿意明确说“材料不足”。

### 10.3.5 PPO 为什么常用于 RLHF

PPO (Proximal Policy Optimization) 的优势在于更新相对稳定，不会让策略一步跳得过远。它常见的裁剪目标可以写成：

$$\mathcal{L}_{\text{PPO}} = \mathbb{E} \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

其中  $r_t(\theta)$  是新旧策略概率比， $\hat{A}_t$  是优势函数。直观理解就是：奖励高的行为可以被加强，但加强幅度必须被限制在一个稳定区间内。

工程上更重要的不是公式本身，而是它表达的约束思想：对齐训练不是让模型“越偏好越好”，而是要在可控范围内逐步挪动策略。

### 10.3.6 KL 约束的意义

在 RLHF 中常加入 KL 约束，目的是避免策略为了追求奖励而完全偏离原有语言能力。工程上这意味着：我们希望模型“更符合偏好”，但不希望它因此失去可读性、事实性和基础任务能力。

如果把 KL 的作用说得更直白一点，它相当于在提醒训练过程：可以更偏向我们喜欢的回答，但不要忘了拿高分把原本会的东西也练坏。

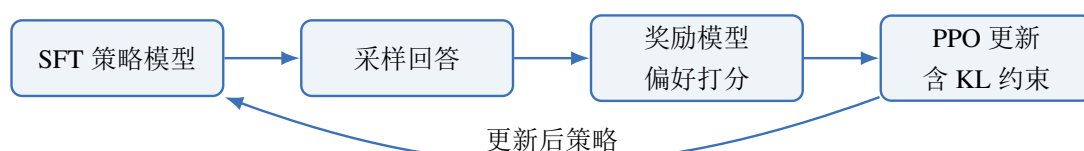


图 10.1: RLHF/PPO 的最小闭环

### 10.3.7 奖励黑客与错配风险

只要奖励模型学得不对，策略模型就可能学会“迎合评分规则”，而不是真正提高可用性。这就是常说的奖励黑客风险。

例如，奖励模型如果过度偏爱长答案，策略模型就可能在材料不足时也写出冗长、肯定、看似负责但实际风险更高的回答。对企业知识助手来说，这种错配尤其危险，因为它会让系统在最该保守的时候反而更会“装懂”。

### 10.3.8 什么时候不该上 RLHF

对很多工程团队来说，更重要的问题不是“如何做 RLHF”，而是“现在到底该不该做”。以下几类场景通常都不应优先进入 RLHF：

- 问题主要来自知识缺失、文档过期或检索失败。
- 连基础 SFT、提示模板和评测集都还不稳定。
- 没有可复用的偏好标注数据，只能凭印象判断“回答更好”。
- 团队无法做回归评测，不知道行为变化是提升还是漂移。



| 当前主要问题          | 更像哪一层问题 | 更优先的动作           |
|-----------------|---------|------------------|
| 答案依赖最新制度，但模型答错  | 知识接入问题  | 先修 RAG 与知识库更新    |
| 回答格式不稳、风格混乱     | 任务行为问题  | 先改 prompt 或做 SFT |
| 多种回答都能答对，但偏好不稳定 | 对齐问题    | 这时才考虑偏好数据与 RLHF  |
| 没有稳定评测集         | 基础工程问题  | 先建评测体系，再谈 RLHF   |

表 10.2: RLHF 不是第一响应动作，而是偏好问题的后手

### 10.3.9 强化学习在 NLP 中的角色

在大语言模型之外，强化学习在 NLP 中还常见于：

- 对话策略优化。
- 长序列决策与工具调用。
- 多步推理中的行为规划。
- 从执行结果或环境反馈中学习。

这提醒我们：RLHF 不是“只有大模型才需要的神秘技术”，它只是把强化学习的一套思想，带到了语言行为优化里。

## 10.4 主案例推进

对于企业知识助手，很多关键行为并不是“事实对错”问题，而是“输出偏好”问题。例如：

- 材料不足时，宁愿保守也不要猜测。
- 优先给出依据，再给建议动作。
- 对高风险流程问题，优先引导人工确认。
- 在格式上保持统一，便于工单系统或知识系统接入。

这些要求可以先通过提示工程和 SFT 处理；只有当系统规模扩大、偏好边界复杂、且已有足够反馈数据时，RLHF 才值得进入路线图。换句话说，对齐训练通常是中后期优化手段，而不是第一步。

如果把企业知识助手的偏好训练路线再写得具体一点，可以是：

1. 先用 SFT 让模型学会固定输出格式和基础任务行为。
2. 再从人工审核记录、失败会话和高风险场景中提取偏好样本。
3. 训练奖励模型，让系统学会区分“更保守、更有依据、更可执行”的回答。
4. 用 PPO 在 KL 约束下做策略更新。
5. 最后用评测集和人工抽检确认：模型真的更稳了，而不是更会迎合奖励。

在这个案例里，真正的难点并不是写出 PPO 代码，而是把“什么叫更好的企业回答”定义清楚。定义不清，对齐训练就会成为高成本的放大器，把原本就模糊的目标放得更模糊。

## 10.5 常见坑与工程取舍

- 把 **RLHF** 当成通用升级按钮：没有稳定偏好数据和评测体系时，引入 RLHF 风险很高。
- 奖励模型目标模糊：奖励定义不清，最终会把模型推向不可解释的行为。
- 忽略 **KL** 约束：模型可能为了局部高奖励而丢失基础语言能力。
- 在错误层面使用 **RL**：如果问题本质是检索失败或文档过期，强化学习无法替代系统修复。  
再补四个更贴近工程的风险：
- 偏好标注标准前后不一：奖励模型学到的是标注员习惯，而不是团队共识。
- 只看离线奖励，不看真实回归样本：模型可能更会拿分，但没更可用。
- 奖励过度偏爱某种表面特征：例如更长、更坚定、更模板化的答案。
- 把对齐训练和安全治理混为一谈：RLHF 能改善偏好，但不能代替权限、审计和规则系统。

## 10.6 本章练习

 **练习 10.1 KL 的作用** 为什么 RLHF 中常常需要加入 KL 约束，而不是只追求奖励越高越好？

**解** 因为如果只追求奖励，模型可能会为了局部高分而偏离原有语言能力和事实性。KL 约束的作用是限制策略更新幅度，让模型在“更符合偏好”和“保持原有可读性、稳定性”之间维持平衡。

 **练习 10.2 奖励黑客风险** 某奖励模型特别偏爱“语气坚定、长度较长”的回答，结果策略模型开始在材料不足时也写出很长的肯定性答案。这说明了什么风险？

**解** 这说明出现了奖励黑客或奖励错配风险。模型学会了迎合奖励模型的表面偏好，而不是真正提高可用性。此时应回查奖励定义、偏好标注标准和回归评测，而不是继续加大 RL 更新强度。

## 10.7 延伸阅读

- **Ouyang et al., *InstructGPT***: 理解 RLHF 在指令模型中的早期经典流程。
- **Schulman et al., *PPO***: 理解 PPO 为什么被广泛用于稳定策略更新。
- **DPO、RLAIF 等后续路线资料**: 适合理解 RLHF 并不是对齐训练的唯一方式。
- **偏好标注与奖励建模实践资料**: 适合理解“奖励定义清楚”为什么比“算法更复杂”更重要。

## 10.8 小结

本章建立了对齐训练的工程坐标：**SFT** 解决“学会任务”，**RLHF** 解决“更符合偏好”，**PPO** 则是常见的稳定优化工具。真正成熟的团队不会盲目上 RLHF，而是先明确偏好目标、奖励来源和回归评测，再决定是否值得走到这一步。

## 10.9 下一章过渡

不论是否进入 RLHF，只要系统准备上线，就必须面对另一个更现实的问题：延迟、显存和吞吐。下一章将进入推理与性能优化，讨论如何让企业知识助手既可用又可部署。

# 第 11 章 推理与性能优化：显存、量化、吞吐与服务稳定性

## 内容提要

- ❑ **本章核心问题：**在真实部署约束下，应该如何在回答质量、延迟、显存和成本之间为企业知识助手做推理侧取舍？
- ❑ **主案例推进：**把企业知识助手从“能回答”推进到“能上线、能控成本、能稳住尾延迟”。
- ❑ **读完后能做什么：**看到“太慢、太贵、太吃显存”时，先找最大头瓶颈，而不是同时乱调所有优化开关。

## 11.1 学习目标

- 理解推理阶段的主要成本来源：参数、KV Cache、批处理与上下文长度。
- 掌握量化、批处理、缓存和上下文预算等常见优化手段。
- 能为企业知识助手设计一套兼顾效果、成本与稳定性的部署策略。

## 11.2 问题背景

到项目中后期，团队最常抱怨的已经不是“模型答得不够好”，而是“太慢、太贵、太吃显存”。推理优化的核心不是把所有数字压到最低，而是在用户体验、回答质量、资源预算和系统稳定性之间找到平衡。

这个阶段最容易出现两种误判。第一种是把所有问题都归到“模型太大”，结果忽略了真正的瓶颈其实是上下文太长、输出太啰嗦或缓存失效。第二种是看到显存或延迟上升就一口气把量化、批处理、缓存、截断和多模型路由全开，最后连哪一项起作用都说不清。推理优化真正需要的不是更多技巧，而是一条从症状到瓶颈、再到动作的诊断顺序。

## 11.3 核心概念

### 11.3.1 推理成本从哪里来

推理成本主要来自四个方面：

1. **模型参数占用：**模型越大，常驻显存越高。
2. **上下文长度：**输入越长，Attention 与 KV Cache 成本越高。
3. **输出长度：**生成越长，单次请求耗时越高。
4. **并发压力：**同时服务多个请求时，吞吐与尾延迟会迅速恶化。

如果把显存来源再拆得更工程化一些，可以写成：

推理显存  $\approx$  模型权重 + KV Cache + 运行时缓冲与中间变量 + 框架开销。

| 显存组成       | 什么时候最明显       | 工程判断                  |
|------------|---------------|-----------------------|
| 模型权重       | 模型一加载就占住      | 与参数量和精度直接相关           |
| KV Cache   | 长上下文、长输出、高并发  | RAG 和多轮对话最容易放大这一项     |
| 运行时缓冲与中间变量 | 大 batch、复杂服务栈 | 往往在高吞吐配置下明显增加         |
| 框架与碎片开销    | 长时间运行、多实例部署   | 需要靠监控和 profiler 才容易发现 |

表 11.1: 推理显存不是一个数字，而是几类开销的叠加

一个足够好用的简化估算方法，是先估算权重显存：**参数量乘以每个参数所占字节数**，再给上下文、KV Cache 和框架余量预留空间。这个估算不需要精确到 MB，但足以帮助团队判断“问题到底出在模型本体，还是出在服务配置”。

11.3.2 量化的意义

量化通过降低权重或激活表示精度来减少显存占用并提升吞吐。常见路线包括 FP16、INT8 和 INT4。工程上应当记住：量化的价值不仅是“省卡”，更是让更多部署形态成为可能，例如单机、多实例或边缘部署。

| 精度方案        | 主要收益          | 主要风险      | 适用判断             |
|-------------|---------------|-----------|------------------|
| FP16 / BF16 | 兼顾稳定性与较高速度    | 显存压力仍较高   | 基线部署、先求稳时使用      |
| INT8        | 明显降显存、通常较稳    | 个别任务精度回退  | 显存开始吃紧但仍需较稳质量时使用 |
| INT4        | 进一步降显存、便于单机部署 | 某些任务质量更敏感 | 资源紧张且必须配合回归测试时使用 |

表 11.2: 不同精度的收益与风险

量化最重要的工程纪律，是**量化前后必须跑同一套评测集**。因为有些任务对精度变化极其敏感，尤其是长文本综合、细粒度分类和格式严格任务。没有回归测试，所谓“省出来的资源”很可能换来的是静悄悄的质量回退。

11.3.3 KV Cache 与上下文预算

对解码器模型来说，长对话和长文档会显著推高 KV Cache 成本。因此推理优化往往不只是模型层问题，也包括上游上下文管理问题。控制无效历史轮次、压缩检索片段和限制输出长度，常常比单纯换部署框架更有效。

从因果链上看，这个问题可以理解为：

- 1. 输入上下文越长，需要缓存的 token 越多。
- 2. 输出越长，缓存会持续累积而不是只在输入阶段出现。

- 3. 并发越高，多个请求的 KV Cache 叠加得越快。
- 4. 当缓存顶到显存边界时，尾延迟、抖动和失败率都会一起上升。

这也是为什么 RAG 系统在“答案质量看起来没怎么变”的情况下，显存和延迟却会突然恶化。真正放大的，不一定是模型，而是上下文预算失控。

11.3.4 吞吐与延迟的基本取舍

批处理通常能提高吞吐，但会拉高单请求等待时间。对内部知识助手来说，如果主要场景是交互式问答，P95 延迟通常比理论最大吞吐更重要；如果场景是离线批量摘要，则可以接受更积极的批处理策略。

| 业务模式      | 优先目标          | 典型策略                    |
|-----------|---------------|-------------------------|
| 交互式问答     | 降 P95 延迟、稳住抖动 | 控上下文、保守批处理、缓存热点问题       |
| 离线批量摘要/抽取 | 提吞吐、降单位成本     | 更积极的批处理、更长队列、更高 GPU 利用率 |
| 混合场景      | 分开服务等级        | 在线与离线任务走不同队列或不同模型配置     |

表 11.3: 交互式问答与离线处理的优化路线并不相同

11.3.5 先定位瓶颈，再挑手段

推理优化最实用的顺序通常不是“先上最热门框架”，而是：

- 1. 先看上下文是否过长，尤其是 RAG 片段和历史对话是否失控。
- 2. 再看输出是否过长，是否缺少明确的生成长度边界。
- 3. 然后判断显存瓶颈是否值得通过量化解决。
- 4. 再看 batch 和并发策略是否让尾延迟被放大。
- 5. 最后才考虑更重的服务框架、offload 或分布式方案。

这条顺序的好处是，前几步通常成本最低、见效最快，而且最不容易引入新的复杂性。

11.4 主案例推进

在主案例中，我们按以下顺序优化部署：

- 1. 先压缩 prompt 和检索片段，控制上下文预算。
- 2. 再尝试量化，评估回答质量是否可接受。
- 3. 根据业务模式选择偏低延迟还是偏高吞吐的服务配置。
- 4. 为热问题引入缓存，为重复文档问题减少重复计算。
- 5. 最后再考虑更复杂的服务框架或多模型路由。

下面这张对照表可以帮助团队把优化动作和代价联系起来：

| 优化手段             | 主要收益     | 主要代价       | 适用判断                |
|------------------|----------|------------|---------------------|
| 压缩上下文            | 降延迟、降显存  | 可能损失依据覆盖   | Prompt 或检索片段明显过长时优先 |
| 限制输出长度           | 降尾延迟、降成本 | 回答可能不够完整   | 回答常常冗长发散时优先         |
| 8-bit / 4-bit 量化 | 降显存、提吞吐  | 可能损失部分任务精度 | 显存是主瓶颈且可接受回归测试时     |
| 批处理              | 提吞吐      | 增加单请求等待    | 离线任务或高并发后台场景更适合     |
| 缓存               | 降重复计算成本  | 需要失效策略     | 高频重复问题且知识变化不快时适合    |

如果团队进一步想知道“瓶颈在哪”，可以从监控和 profiler 角度用三种方法建立直觉：

- **吞吐量估计**：看单位时间能处理多少请求或多少 token，适合快速横向对比。
- **理论算力对比**：看理论 FLOPS 与实际吞吐的差距，适合判断 GPU 是否吃满。
- **Profiler 分析**：看算子时间、内存分配、I/O 和等待，适合做真正的深挖。

更实用的是把常见症状直接映射成第一检查项：

| 现象               | 最该先怀疑什么              | 第一动作                  |
|------------------|----------------------|-----------------------|
| RAG 上线后显存暴涨      | 上下文长度与 KV Cache      | 先缩检索片段和历史轮次           |
| 平均延迟还行，但用户抱怨忽快忽慢 | 尾延迟和队列抖动             | 看 P95/P99 与 batch 策略  |
| 显存够，但吞吐上不去       | GPU 未吃满、I/O 或小 batch | 做 profiler，检查预填充与解码阶段 |
| 量化后成本下降但用户投诉变多   | 精度回退                 | 立刻跑固定评测集和关键样本回放       |

表 11.4: 从症状到第一检查项的推理排障顺序

这条路线体现了一个重要原则：推理优化首先是系统预算管理问题，其次才是框架调优问题。

## 11.5 常见坑与工程取舍

- **只优化模型，不优化输入**：很多高延迟问题本质上是 prompt 过长。
- **量化前不做回归测试**：某些任务对精度变化很敏感，必须用评测集验证。
- **只看平均延迟**：用户更容易感知 P95 或 P99 的抖动。
- **把缓存当万灵药**：回答依赖动态知识时，缓存失效策略必须非常明确。

再补四条部署期常见误区：

- **显存问题只看模型大小**：很多时候真正失控的是 KV Cache，而不是权重。
- **交互式系统套离线优化思路**：吞吐提上去了，用户体验却变差了。
- **忽略内存碎片和框架开销**：长时间运行后，纸面估算和真实表现可能差很多。
- **一味追求满卡率**：GPU 利用率高不代表整体服务质量高，稳定性和尾延迟同样重要。



## 11.6 本章练习

 **练习 11.1 显存瓶颈定位** 某企业知识助手在模型不变的情况下，随着 RAG 上线后显存占用显著升高。你应先怀疑哪一类因素？

**解** 应先怀疑上下文长度和 KV Cache 成本，因为 RAG 上线后最常见的变化是带入了更多检索片段，推高了输入长度。此时先压缩上下文通常比先换推理框架更有效。

 **练习 11.2 量化还是批处理** 如果系统主要是员工实时问答，而不是离线批量生成，面对成本压力时为什么通常先考虑量化和上下文压缩，而不是激进批处理？

**解** 因为实时问答更敏感的是单请求延迟和尾延迟。激进批处理虽然能提吞吐，但会增加等待时间，损害交互体验。量化和上下文压缩更容易在不明显牺牲响应性的前提下降低成本。

## 11.7 延伸阅读

- vLLM、TensorRT-LLM、TGI 等推理框架文档：适理解不同服务栈怎样影响吞吐与延迟。
- bitsandbytes、AWQ、GPTQ 等量化资料：适理解量化为何会改变显存与质量平衡。
- 各类生产系统性能调优案例：适理解 P95、P99 和缓存策略在真实部署中的重要性。
- PyTorch Profiler 与 GPU 监控实践：适理解瓶颈定位如何从“猜问题”升级成“量问题”。

## 11.8 小结

推理优化的核心在于预算：预算显存、预算上下文、预算输出长度、预算并发。成熟系统不是把单一指标做到极致，而是让效果、成本和稳定性形成可接受平衡；而做到这一点的前提，是先把瓶颈定位对。

## 11.9 下一章过渡

当单机推理和局部优化都触顶后，团队还会遇到更大的训练与服务规模问题。下一章将收束到分布式训练与大规模工程，讨论如何在更大算力和更复杂系统中继续推进模型能力。



## 第 12 章 分布式训练与大规模工程：并行策略、训练框架、成本与排障

### 内容提要

- ❑ **本章核心问题：**当企业知识助手背后的训练和实验规模继续增长时，团队应该如何选择并行策略、训练框架和排障方法，才能让工程演进可持续？
- ❑ **主案例推进：**把企业知识助手的技术路线从单机实验收束到平台化、规模化的大模型工程。
- ❑ **读完后能做什么：**面对单卡放不下、通信开销过大、实验难复现等问题时，知道应先选哪类并行策略和基础设施。

### 12.1 学习目标

- 理解数据并行、张量并行、流水线并行与参数分片的基本职责。
- 掌握训练框架选型、成本评估和常见排障思路。
- 为企业知识助手背后的模型工程建立规模化视角。

### 12.2 问题背景

单卡和单机足以支撑很多原型系统，但当团队开始尝试更大模型、更长上下文、更大批次或继续预训练时，资源瓶颈就会迅速暴露。分布式工程的本质不是“把训练拆到多张卡上”这么简单，而是要重新分配计算、显存、通信和失败恢复的责任。

这也是很多团队第一次真正感受到“大模型工程”和“单机深度学习实验”的差别。单机实验里，问题往往是 loss 不降、显存不够或脚本报错；进入分布式后，新的问题变成通信等待、拓扑不合适、检查点写不动、日志无法归因，甚至是“模型其实没坏，但系统层面已经不可维护”。因此，本章真正要建立的是一种规模化工程视角：参数增长只是开始，系统复杂度才是后半程的主角。

### 12.3 核心概念

#### 12.3.1 通信基础：点对点与集体通信

在讲并行策略之前，先建立一个最小通信视角。分布式训练里的通信大体可以分成两类：

- **点对点通信：**更常见于流水线并行，相邻阶段传递激活和梯度。
- **集体通信：**更常见于数据并行和张量并行，例如 All-Reduce、Reduce-Scatter、All-Gather。

这一区分很重要，因为它直接决定瓶颈会落在哪。流水线并行更容易遇到阶段不均衡与等待，数据并行更容易被梯度同步拖住，张量并行则更依赖高带宽、低时延的通信环境。

| 通信类型  | 典型出现位置          | 团队最该关注什么           |
|-------|-----------------|--------------------|
| 点对点通信 | 流水线并行 PP        | 相邻阶段是否均衡、是否出现等待和气泡 |
| 集体通信  | 数据并行 DP、张量并行 TP | 同步代价是否吞掉了多卡带来的收益   |

表 12.1: 并行策略背后对应的通信模式

### 12.3.2 数据并行

数据并行把同一个模型复制到多张卡上，每张卡处理不同批次数据，再同步梯度。它实现简单，是很多训练任务的起点，但显存压力仍然存在，因为每张卡都要持有完整模型。

数据并行的优势在于概念简单、生态成熟，但真正把它跑快，常常依赖三个工程抓手：

- **梯度分桶**：减少同步时的碎片化开销。
- **计算通信重叠**：让反向传播和梯度同步尽量并行发生。
- **梯度累积**：在显存受限时，用更多步数换更大等效 batch。

这三点都说明了一件事：数据并行并不只是“多卡复制模型”，而是一整套如何减少同步浪费的实践。

### 12.3.3 张量并行

张量并行把单层中的大矩阵切到多张卡上，适合模型宽度很大、单卡根本放不下的场景。它能解决单卡显存瓶颈，但通信要求更高。

更准确地说，张量并行解决的是“单层算子本身太大”的问题，而不是一般意义上的“模型太大”。如果主要矛盾只是整体显存不够，很多时候先用参数分片就够了；只有当单层矩阵真的大到必须切开时，张量并行才会成为主角。

### 12.3.4 流水线并行

流水线并行把不同层分配到不同设备上，像装配线一样前向与反向流动。它适合模型很深的场景，但容易出现流水线气泡，需要通过 micro-batch 设计来缓解。

旧稿里关于 GPipe、PipeDream 和 1F1B 的内容，教材化后最值得保留的，不是具体论文细节，而是一个判断框架：

- **GPipe 类思路**：更容易理解，但阶段空转较明显。
- **PipeDream / 1F1B 类思路**：通过更积极的调度减少等待，但实现和状态管理更复杂。

工程上最重要的直觉是：流水线并行不是白送的并行度，它是用调度复杂性去换更大模型可训练性。

### 12.3.5 参数分片与内存优化

像 ZeRO、FSDP 这类方案，通过对参数、梯度和优化器状态做分片，显著降低单卡显存占用。工程上它们的重要性在于：很多时候团队并不需要“更大集群”，而只需要更聪明的内存分配方式。

| 方案阶段             | 主要分片什么          | 主要收益        | 主要代价       |
|------------------|-----------------|-------------|------------|
| ZeRO / FSDP 基础阶段 | 优化器状态           | 先减一部分训练常驻内存 | 收益有限，适合先入门 |
| 更深一层的分片          | 优化器状态 + 梯度      | 显著降训练显存     | 同步逻辑更复杂    |
| 最激进的分片           | 参数 + 梯度 + 优化器状态 | 单卡压力降得最多    | 通信与实现复杂度最高 |

表 12.2: 参数分片方案的阶段化理解

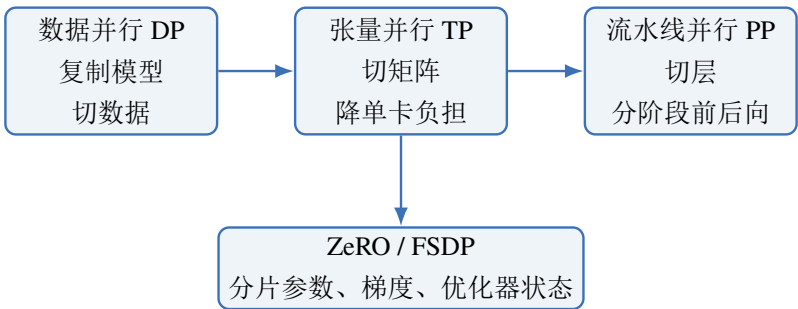


图 12.1: 大规模训练中常见的并行与分片策略

12.3.6 框架选型思路

对工程团队来说，训练框架的选择通常围绕四个问题：

- 是否与现有模型生态兼容。
- 是否容易调试和复现实验。
- 是否支持所需并行策略。
- 是否有足够稳定的社区和运维经验。

| 框架/工具          | 更适合什么阶段              | 主要优势            | 主要提醒           |
|----------------|----------------------|-----------------|----------------|
| Accelerate     | 快速原型、较轻训练任务          | 上手快、屏蔽很多分布式细节   | 不是所有极限规模场景都最合适 |
| FSDP           | 通用多卡训练、原生 PyTorch 生态 | 原生、稳定、参数分片清晰    | 仍需要较强的配置理解     |
| DeepSpeed      | 更激进的显存优化和大规模训练       | ZeRO 生态成熟、可扩展性强 | 配置复杂，调试成本更高    |
| Megatron-LM 类栈 | 超大模型、TP/PP 需求明显时     | 针对大规模并行优化深入     | 学习成本和工程门槛更高    |

表 12.3: 训练框架不是“谁最强”，而是“谁最适配当前阶段”

12.3.7 并行策略不是越复杂越好

在很多团队里，分布式方案会被误解成“越先进越好”。实际上，更合理的顺序通常是：

- 单卡能放下，就先别拆并行。
- 单卡放不下但多卡通信还可控，优先考虑数据并行加参数分片。
- 只有当单层矩阵或模型深度真的成为硬瓶颈时，再上张量并行或流水线并行。
- 只有当模型规模、网络条件和框架成熟度同时允许时，才考虑更重的 3D 并行组合。

## 12.4 主案例推进

对企业知识助手而言，并不是一开始就需要大规模训练，但团队应当提前理解升级路径：

1. **原型期**：单机推理、少量微调实验。
2. **成长期**：多卡 LoRA/QLoRA、系统化评测和持续数据迭代。
3. **平台期**：继续预训练、参数分片、多任务适配器管理、统一实验平台。

一旦进入平台期，真正重要的就不只是模型本身，而是整条工程链路：数据版本、检查点管理、日志系统、失败恢复、成本统计和实验复现。

对一个准备迈向平台期的团队，下面这份检查清单比“会不会写分布式脚本”更关键：

- **弹性容错**：训练中断后能否自动恢复。
- **自动重启**：单点失败时是否需要人工盯守。
- **检查点周期**：保存太稀，失败代价高；保存太频，I/O 成本高。
- **实验记录**：超参数、数据版本、代码版本和评测结果是否能对上。
- **成本核算**：一次实验用了多少 GPU 小时、产出了多少有效样本速度。

| 成本视角   | 应该记录什么           | 为什么重要         |
|--------|------------------|---------------|
| GPU 小时 | 一次实验实际消耗多少卡时     | 决定方案是否值得继续    |
| 吞吐     | 每秒处理多少 token 或样本 | 评估并行策略是否真的生效  |
| 失败恢复成本 | 中断后回到可继续状态要花多久   | 决定平台期稳定性      |
| 有效样本速率 | 不是只看跑了多久，而是学到了多少 | 防止“算力很满，收益很低” |

表 12.4: 大规模工程里，成本核算本身就是工程能力

## 12.5 常见坑与工程取舍

- **先上复杂并行，再补可观测性**：一旦训练失败，几乎无法定位问题。
  - **忽略通信成本**：多卡不一定更快，通信瓶颈可能吞掉全部收益。
  - **只按论文配置抄框架**：真实环境中的网络、驱动和调度器差异很大。
  - **没有成本核算**：如果不知道一次实验花了多少 GPU 小时，就无法判断方案是否值得继续。
- 再补一份更贴近一线排障的最小手册：
- **多机起不来**：先查环境变量、端口、NCCL 配置与节点间连通性。
  - **吞吐扩不起来**：查通信是否压过计算，查 batch 是否太小，查数据加载是否卡住。

- **首卡显存明显更高**：查模型初始化、日志、保存与额外控制逻辑是否都落在同一张卡。
- **checkpoint 特别慢**：查存储带宽和保存频率，不要只怪训练框架。
- **训练偶发 NaN 或 loss 异常**：查混合精度、学习率、恢复训练时的状态是否完整。

## 12.6 本章练习

🔥 **练习 12.1 并行策略选择** 如果模型刚好单卡放不下，但团队的多卡通信条件一般，此时通常应优先考虑哪类方案？

**解** 通常应优先考虑数据并行配合参数分片，例如 ZeRO 或 FSDP，而不是立刻上更复杂的张量并行或流水线并行。因为当前主要矛盾是显存装不下，而不是单层计算结构本身必须被切开。

🔥 **练习 12.2 训练故障归因** 为什么大规模训练里“日志、检查点和成本统计”不是附属品，而是核心基础设施？

**解** 因为规模一旦上来，失败代价会迅速变大。没有日志就难以定位故障层，没有检查点就难以恢复训练，没有成本统计就无法判断实验是否值得继续。平台期的大模型工程，本质上是可复现、可恢复、可核算的系统工程。

## 12.7 延伸阅读

- **DeepSpeed、FSDP、Megatron-LM 官方文档**：适合理解主流并行策略各自解决什么瓶颈。
- **PyTorch Distributed 文档**：适合理解数据并行和分布式基础接口。
- **各类大模型训练技术报告中的工程章节**：适合理解真实团队如何在成本、通信和稳定性之间做取舍。
- **NCCL、集群拓扑与 profiler 实践资料**：适合理解为什么“能跑”和“跑得好”之间差着一整套系统工作。

## 12.8 小结

分布式工程是大模型能力真正工业化的最后一层基础设施。它让训练、继续预训练和大规模实验成为可能，也把“模型优化”扩展成“系统优化”。当团队走到这一步，决定成败的往往不再是某个单独技巧，而是整套并行、监控、恢复和核算体系是否站得住。

## 12.9 下一章过渡

作为全书最后一章，这里的“过渡”不再通向新的主题，而是通向读者自己的项目实践。至此，本书完成了从模型原理到企业知识助手落地的完整主线：先理解模型，再学会调用与控制模型；接着通过微调、数据工程和 RAG 把知识系统搭起来；之后用评测、对齐和推理优化让系统稳定可用；最后再把视角提升到分布式训练与大规模工程。